

A Numerically Exact Algorithm for the Bin-Packing Problem

R. Baldacci, S. Coniglio, F. Furini

Abstract

We propose a numerically-exact algorithm for solving the Bin Packing Problem (BPP) based on a branch-price-and-cut framework combined with a pattern-enumeration method. Key to the algorithm is a novel technique for the computation of numerically-safe dual bounds for the widely-adopted set covering reformulation of the BPP (tightened with additional valid inequalities) with a precision that is higher than the one of general-purpose floating-point solvers. Our branch-price-and-cut algorithm also relies on an exact integer (fixed-point) label setting algorithm for solving the pricing problem associated with the tightened set covering formulation. To the best of our knowledge, ours is the first algorithm for the BPP that is numerically exact and practical for solving large-scale instances. Extensive computational results on instances affected by notorious numerical difficulties (those of the Augmented Non-IRUP class) show that our exact algorithm outperforms all the not numerically-exact state-of-the-art algorithms based on branch-and-cut-and-price techniques that rely on a set-covering formulation of the BPP.

Keywords: bin packing, numerical precision, branch-price-and-cut algorithm, dynamic programming.

1 Introduction

Given a set N of n items, a positive integer weight w_j associated with each item $j \in N$, and an unlimited number of identical bins of integer capacity W , the *Bin Packing Problem* (BPP) asks for finding the minimum number of bins that are needed to pack all the items. The BPP is one of the fundamental problems in Combinatorial Optimization and Operations Research and, while it has been extensively studied for decades, it is still very challenging to solve to optimality. The problem occurs in countless areas ranging from job scheduling to logistics and in very diverse applications such as truck or container loading with weight capacity constraints, digital content management in cloud storage, and event seating. For more details, we refer the reader to the survey by [Delorme et al. \(2016\)](#).

Most of the state-of-the-art algorithms for the BPP are based on solving a Set Covering (SC) formulation featuring an exponential number of variables associated with subsets of items of total weight no larger than W , called *patterns*. Letting $\mathcal{P} = \left\{ S \subseteq N : \sum_{j \in S} w_j \leq W \right\}$ be the collection of all patterns and letting the binary variable ξ_S be equal to 1 if and only if pattern

$S \in \mathcal{P}$ is featured in the solution, the SC formulation reads

$$\begin{aligned}
(\text{SC}) \quad & \min \sum_{S \in \mathcal{P}} \xi_S & (1a) \\
& \text{s.t.} \sum_{S \in \mathcal{P}(j)} \xi_S \geq 1 & j \in N & (1b) \\
& \xi_S \in \{0, 1\} & S \in \mathcal{P}, & (1c)
\end{aligned}$$

where $\mathcal{P}(j) \subseteq \mathcal{P}$ is the collection of patterns containing item $j \in N$.

Let LSC be the Linear Programming (LP) relaxation of the SC formulation. The problem underlying this relaxation is usually called the Fractional Bin Packing Problem (FBPP); see, e.g., [Karmarkar and Karp \(1982\)](#); [Eisenbrand et al. \(2013\)](#); [Caprara et al. \(2015\)](#); [Coniglio et al. \(2019\)](#). Let $z(\text{LSC})$ be the optimal solution value of LSC and let $z(\text{BPP})$ be the optimal solution value of the BPP. Any BPP instance such that $z(\text{BPP}) = \lceil z(\text{LSC}) \rceil$ is said to enjoy the *Integer Round-Up Property* (IRUP) ([Kartak et al., 2015](#); [Delorme et al., 2016](#)). While this property holds for many BPP instances, several non-IRUP instances are known in the literature ([Caprara et al., 2015](#)). The *Modified Integer Round-Up Property* (MIRUP) is a weaker version of the IRUP that holds if either $z(\text{BPP}) = \lceil z(\text{LSC}) \rceil$ or their difference is no larger than 1 (i.e., $z(\text{BPP}) - \lceil z(\text{LSC}) \rceil \leq 1$). The MIRUP has been proven to hold when the number of different item sizes is no larger than 7 (see [Eisenbrand et al. \(2013\)](#)), and it is conjectured to hold in general ([Scheithauer and Terno, 1997](#)).

While, on the IRUP instances, computing $z(\text{LSC})$ suffices to certify (or disprove) the optimality of a given solution, non-IRUP instances require more sophisticated methods to produce a strong bound to do so.

This situation is exacerbated by the fact that classical approaches for solving the SC formulation of the BPP are known to suffer from numerical errors on certain classes of non-IRUP instances when floating-point solvers are used. The Augmented Non-IRUP (ANI) instances proposed by [Delorme et al. \(2016\)](#) are one such class. As noted by [Pessoa et al. \(2020, 2021\)](#), these numerical errors may lead to computing incorrect bounds. Such an event can be catastrophic for the exactness of the solution algorithm, as it may lead to constructing an invalid enumeration tree as a result of the application of invalid pruning operations, which would constitute an invalid proof of optimality of the solution that was found.

In the majority of the state-of-the-art algorithms for the BPP, the SC formulation is solved with a Branch-Price-and-Cut (BPC) algorithm as originally proposed by [Vance et al. \(1994\)](#). At each node of the BPC tree, the LP relaxation is solved by a column-and-row-generation method following the original work of [Gilmore and Gomory \(1961\)](#). In it, the patterns are generated by solving a pricing problem which, at the root node of the BPC tree, coincides with an instance of the Knapsack Problem (KP). With only a few exceptions (see the works of [Belov and Scheithauer \(2006\)](#) and of [Pessoa et al. \(2020, 2021\)](#)), branching is carried out following the classical rule proposed by [Ryan and Foster \(1981\)](#), according to which two items are forced to be packed into the same bin or into different ones.

A relatively new addition to the state of the art is strengthening the SC formulation with additional *master cuts* such as the *subset-row inequalities of cardinality three* (SR3s). SR3s are a family of rank-1 Chvátal-Gomory cuts introduced into the BPP literature by [Wei et al. \(2020\)](#) and first proposed by [Jepsen et al. \(2008\)](#) that are valid for set partitioning as well as for set covering problems such as (1) where Constraints (1b) are tight w.l.o.g. in any optimal solution.

Another new addition to the state of the art is *pattern enumeration*. Originally proposed by [Baldacci et al. \(2008\)](#) for the Vehicle Routing Problem (VRP) and used in the context of the BPP by [Pessoa et al. \(2021\)](#), the technique coincides with the enumeration of all patterns whose reduced cost is no larger than the gap between the value of the incumbent and $z(\text{LSC})$ (possibly strengthened by master cuts). Such an enumeration leads to building a *reduced SC problem* which can be solved to optimality by a branch-and-bound method without generating any further patterns.

For more details on the state-of-the-art BPC methods for the BPP, we refer the reader to [Wei et al. \(2020\)](#) for the EXM algorithm and to [Pessoa et al. \(2020, 2021\)](#) for the general-purpose BPC algorithm VRPSolver, which can tackle variants of the VRP (of which the BPP is a special case).

For another family of methods (the most recent one being the NF-F algorithm) we refer the reader to [de Lima et al. \(2022\)](#). Such methods rely on a *network-flow framework* designed to solve the integer counterpart to a Dantzig-Wolfe decomposition via a path-flow and arc-flow model, and can be adapted to solve instances of the BPP. In particular, such methods rely on arc-flow formulations of pseudo-polynomial size which enjoy strong linear relaxations. Key to the efficiency of NF-F is a novel asymmetric branching scheme where a collection of smaller arc-flow subproblems are solved by a general Mixed Integer Linear Programming (MILP) solver. Branching is performed by selecting a subset of arcs and by either imposing all of them to be absent from the solution or by imposing that at least one be contained in it.

Crucially, all the state-of-the-art algorithms we mentioned (EXM, VRPSolver, and NF-F) rely either on numerically-unsafe bounds computed by a floating-point LP solver combined with a floating-point pricing algorithm (this is the case of EXM) or on the adoption in a key step of the algorithm of a floating-point MILP solver (this is the case of both VRPSolver and NF-F: in the former, the reduced SC problem is solved as a MILP with the floating-point solver Cplex, whereas in the latter the left child node generated after a branching operation is always solved as an arc-flow MILP formulation with the floating-point solver Gurobi). As a consequence, none of these algorithms is guaranteed to produce numerically-exact solutions, i.e., solutions whose enumeration tree constitutes a mathematically-exact proof of optimality.

1.1 Review on numerically-safe approaches in mixed integer linear programming

The solution methods presented in the mixed integer linear programming literature to avoid numerical inaccuracies inherent to the floating-point computations are based on either a *pure rational* approach, a *safe floating point* approach, or a *hybrid* approach. In addition, the methods

can also be classified based on the type of mathematical formulations they employ: general compact MILP formulations or extended formulations, the latter being characterized by an exponential number of variables.

In the pure rational approach (Applegate et al., 2007; Wunderling, 1996; Gleixner et al., 2012, 2016), all arithmetic operations are performed over rational numbers. This line of research resulted in rational LP solvers such as QSOpt_ex (Applegate et al., 2007) and SoPlex (Wunderling, 1996; Gleixner et al., 2012, 2016) for the exact solution of LP problems.

Regarding *safe floating point* approaches dealing with compact MILP formulations, Cook et al. (2009), Cornuéjols et al. (2013), and Fukasawa and Goycoolea (2011) addressed numerical safety in the context of cutting plane generation, whereas Steffy and Wolter (2012) and Neumaier and Shcherbina (2004) tackled the issue of obtaining safe bounds for MILPs.

The works of Cook et al. (2011, 2013) and Eifler and Gleixner (2022) deal with the design of exact MILP solvers based on hybrid approaches that combine exact rational LP solvers and safe floating point dual bounds. In particular, Eifler and Gleixner (2022) recently proposed a revision and extension of the solution framework proposed by Cook et al. (2013) that integrates symbolic presolving, an exact repair step for solutions obtained from primal heuristics, and a rational LP solver based on LP iterative refinement. Their computational study on the MIPLIB 2017 benchmark shows a significantly improved performance.

To the best of our knowledge, the two only works addressing the computation of safe dual bounds in the context of extended formulations are by Held et al. (2012) and Fukasawa and Poirrier (2017). The work of Held et al. (2012) introduces numerically-safe bounds within a column-generation-based framework in the context of the graph coloring problem. Fukasawa and Poirrier (2017) propose five methods to obtain safe dual bounds for the LP relaxations of the Capacitated VRP (CVRP). The first method is based on the scaling approach proposed by Held et al. (2012). The next three methods are derived from ideas by Applegate et al. (2007) for the traveling salesman problem and adapted to accommodate a column-generation scheme for the CVRP. The last method is based on a Lagrangian relaxation that is specific to the CVRP formulation. Both Held et al. (2012) and Fukasawa and Poirrier (2017) discuss safe floating-point implementations of the dynamic programming procedure employed in the pricing step to obtain a numerically-exact pricing algorithm.

Notably, even if we restricted ourselves to a BPC method designed to employ an adaptation of the scaling technique of Held et al. (2012) as done by Pessoa et al. (2021) (see Proposition 3.2 in this paper for a generalization), we would obtain bounds that are worse than their infinite-precision counterpart by an additive error proportional to the number of variables and the fixed-point precision of the pricing algorithm—see Equation (2) in Held et al. (2012). In some instances, such an error can be non-negligible and produce weak bounds that can lead to prohibitively large search trees. This is, in particular, the case of the ANI instances, where (as we will show in this paper) having access to a numerically-safe dual bound obtained with a precision higher than the one offered by commercial LP solvers can lead to enormous computational advantages.

1.2 Contributions of this paper

In this paper, we propose a novel technique for the computation of *numerically-safe* LP bounds obtained with an *extended numerical precision*, i.e., a precision higher than the one of state-of-the-art commercial solvers. The contributions of this paper can be summarized as follows:

- We present the first numerically-exact algorithm for the BPP that is practical for solving large-scale instances.
- We propose a technique based on the combined adoption of a rational (infinite-precision) LP solver (SoPlex, Gleixner et al. (2016)) with a faster, floating-point one (Gurobi) and on an exact integer (fixed-point) pricing algorithm that we develop *ex novo* for this work. To the best of our knowledge, rational solvers have never been successfully used before in the context of a column generation method.
- We design a new exact integer (fixed-point) pricing algorithm which relies on, among other features, a new fathoming rule; the algorithm is designed in such a way that it can also perform *pattern enumeration*; differently from not numerically-safe approaches such as the one of Pessoa et al. (2021), after the enumeration phase we solve the *reduced SC problem* to optimality by a further call to a simplified version of our (numerically-exact) BPC algorithm in which the pricing algorithm is disabled, which guarantees the lack of numerical errors.
- Thanks to extensive computational results, we validate the effectiveness of our numerically-exact algorithm as well as the relevance of its components on BPP instances affected by notorious numerical difficulties.
- The framework on which our algorithm is based can be of interest for finding numerically-exact solutions to a large array of combinatorial optimization problems that are typically tackled via a BPC method featuring a set covering/partitioning problem as the master problem.

The remainder of the paper is structured as follows. In Section 2, we introduce the BPC method that forms the basis of our algorithm. Section 3 presents the techniques we propose for the computation of extended-precision numerically safe dual bounds and describes the corresponding algorithm based on column-and-row-generation. Section 4 describes our exact integer (fixed-point) label-setting algorithm designed to solve the pricing problem and illustrates how it can be tailored to perform pattern enumeration. Section 6 presents and analyzes the results of our algorithm on a set of BPP instances from the literature. Section 7 concludes the paper and indicates future research directions.

2 Branch-price-and-cut method

We outline in this section the BPC method that forms the basis for our numerically-exact algorithm.

Let $\mathcal{T} = \{T \subseteq N : |T| = 3\}$ be the collection of all the triplets of items in N and, for each $T \in \mathcal{T}$, let $\mathcal{P}(T) = \{S \in \mathcal{P} : |S \cap T| \geq 2\}$ be the subset of patterns with at least two items in T . The SR3 constraints (Jepsen et al., 2008) read

$$(SR3) \quad \sum_{S \in \mathcal{P}(T)} \xi_S \leq 1, \quad T \in \mathcal{T}.$$

We refer to the version of the LSC formulation (the LP relaxation of the SC formulation) tightened by these constraints as LSCT, where “T” stands for “triplets”, and call $z(\text{LSCT})$ its optimal solution value.

The value of $z(\text{LSCT})$ is computed by a column-and-row-generation method. At each iteration, the restricted master problem (RMP)—a formulation featuring only a subset of the variables and constraints of LSCT—is solved with the primal simplex algorithm to obtain a primal and a dual solution. Then, a *column generation* step takes place, and the pricing problem is solved in order to find negative reduced cost columns. If any are found, they are added to the RMP and a new column generation iteration is carried out. If not, a *row generation* step takes place, and SR3 constraints are separated in a cutting plane fashion and added to the RMP. The cutting plane algorithm terminates when no violated SR3s are found, after which one or more iterations of the column-and-row generation method takes place. The computations stop when both the column- and the row-generation algorithms terminate without finding new columns or rows. Due to the small cardinality of the set $\mathcal{T}$, the SR3 inequalities can be separated by complete enumeration. The generation of columns requires, on the contrary, an efficient *ad hoc* pricing algorithm, which we describe in Section 4.1.

Let us consider a node of the BPC tree in which Ryan-Foster (RF) (Ryan and Foster, 1981) branching is applied. According to RF branching, a node with a fractional solution is split into two new nodes by imposing that a chosen pair of items (the RF pair) either be packed in the same bin (“same branch”) or into different bins (“different branch”). In the same-branch case, the pair of items is merged into a *superitem* with a weight equal to the sum of the weights of the individual items and every already-generated pattern that contains only one of the two items is discarded. In the different-branch case, a *conflict* is introduced between the two items to prevent the selection of patterns that contain both of them and every already-generated pattern that violates such a constraint is discarded. While in the pricing problem the same-branch case is handled by simply considering the superitem *in lieu* of the two individual items, in the different-branch case the new conflict constraint must be enforced explicitly when a new pattern is created (see Section 4.1 for more details).

We denote by MP the master problem associated with the node at hand and by $z(\text{MP})$ its optimal solution value. Let also RMP be the corresponding restricted master problem. Let $\overline{\mathcal{T}} \subseteq \mathcal{T}$ be the set of triplets whose SR3 constraints have been generated so far and let $\overline{N} = \{1, 2, \dots, \bar{n}\}$ be the set of $\bar{n}$ items (possibly involving superitems) with integer weights $\bar{w}_j$ for all $j \in \overline{N}$ corresponding to the node. Let $G = (\overline{N}, E)$ be a conflict graph whose edges represent conflicts between the items. Let $\overline{\mathcal{P}}$ be the set of patterns generated so far. For each $j \in \overline{N}$, let $\pi_j \geq 0$

be the dual variable corresponding to Constraint (1b) associated with item j and let $\rho_T \leq 0$ be the dual variable corresponding to the SR3 constraint associated with a triplet T belonging to $\overline{\mathcal{T}}$. For each pattern S , let $\overline{\mathcal{T}}(S) = \{T \in \overline{\mathcal{T}} : |T \cap S| \geq 2\}$ be the set of triplets in $\overline{\mathcal{T}}$ with at least two items in S . The LP dual of the RMP reads:

$$\text{(RMPD)} \quad \max \sum_{j \in \overline{N}} \pi_j + \sum_{T \in \overline{\mathcal{T}}} \rho_T \quad (2a)$$

$$\text{s.t.} \quad \sum_{j \in S} \pi_j + \sum_{T \in \overline{\mathcal{T}}(S)} \rho_T \leq 1 \quad S \in \overline{\mathcal{P}} \quad (2b)$$

$$\pi_j \geq 0 \quad j \in \overline{N} \quad (2c)$$

$$\rho_T \leq 0 \quad T \in \overline{\mathcal{T}}. \quad (2d)$$

The pricing problem of the RMP coincides with an instance of the *Knapsack Problem (KP) with Triplets and Conflicts* (KP-T-C). Its nonempty solutions coincide with the following set of patterns:

$$\widehat{\mathcal{P}} = \left\{ S \subseteq \overline{N} : S \neq \emptyset, \sum_{j \in S} \overline{w}_j \leq W, \text{ and } \{j_1, j_2\} \notin E \text{ for all } j_1, j_2 \in S \right\}.$$

The reduced cost of a pattern $S \in \widehat{\mathcal{P}}$ with respect to a dual solution $(\boldsymbol{\pi}, \boldsymbol{\rho})$ and a parameter b (equal to 1 here—different values will be used later on in the paper) is

$$\bar{c}_S(\boldsymbol{\pi}, \boldsymbol{\rho}, b) = b - \sum_{j \in S} \pi_j - \sum_{T \in \overline{\mathcal{T}}(S)} \rho_T. \quad (3)$$

The KP-T-C asks for a pattern $S' \in \widehat{\mathcal{P}}$ of minimum reduced cost. If $\bar{c}_{S'}(\boldsymbol{\pi}, \boldsymbol{\rho}, 1) < 0$, the pattern is added to the RMP; otherwise, the solution $(\boldsymbol{\pi}, \boldsymbol{\rho})$ is proven to be a feasible dual solution to the MP.

3 Extended-precision numerically safe dual bounds and two-phase column generation method

One of the main goals of this paper is computing a numerically-safe dual bound with a precision that is higher than the one allowed by floating-point LP solvers by relying on both a floating-point LP solver and an infinite-precision (rational) one.

To achieve this, we introduce a procedure that extends the dual-variable scaling technique of [Held et al. \(2012\)](#) (which applies to an RMP with nonnegative dual variables) to the case where the RMP also features dual variables that are nonpositive. Since floating-point solvers naturally approximate the value of the dual variables of any LP (including the RMP) by floating-point numbers, a procedure of this nature is *necessary* as, if not handled with care, these floating-point numbers may lead to two issues: (i) incorrect calculations within the pricing algorithm and (ii) the generation of a column which would not enter the basis due to its reduced cost being

smaller than the optimality tolerance of the solver. In the algorithm we propose, both issues are circumvented by relying on the extended scaling procedure described in this section and by combining it with an exact integer (fixed-point) pricing algorithm (described in Section 4.1).

Our column-and-row-generation method works in two phases. In the first phase, at each iteration the RMP is reoptimized via the floating-point LP solver and the scaling procedure is combined with the pricing algorithm to generate patterns with a reduced cost that lies within the floating-point optimality tolerance of the solver. We refer to the latter as ϵ_0 .

The second phase starts as soon as the pricing algorithm finds a pattern S' whose reduced cost is strictly smaller (in absolute value) than ϵ_0 . Since, even if we assumed that the floating-point solver worked in infinite precision, i.e., with a tolerance $\epsilon_0 = 0$, without incurring any numerical errors, such a small reduced cost would prevent it from bringing the corresponding $\xi_{S'}$ variable into the basis when reoptimizing the RMP, at the beginning of this phase we switch to the rational (infinite-precision) solver and use it until the column-and-row-generation procedure halts (halting is guaranteed by Proposition 3.1 below). The second phase also starts in the event that, even though the reduced cost is not smaller (in absolute value) than ϵ_0 , the floating-point solver failed to bring the newly generated pattern into the basis due to numerical errors incurred within the reduced-cost calculations.

Such a two-phase method allows us to compensate for the weaker dual bound (illustrated in Proposition 3.2) that the scaling procedure inevitably leads to. With it, we achieve a higher numerical precision than the floating-point solver would allow for while also benefiting for many iterations from its higher efficiency. This way, we limit the number of calls to the computationally more onerous rational solver and, overall, achieve a good trade-off between numerical precision and computational efficiency without forsaking numerical exactness.

3.1 Extended dual scaling technique for the computation of numerically-safe dual bounds

Let $(\pi^{\text{float}}, \rho^{\text{float}})$ be a floating-point dual solution to the RMP of value z^{float} computed by a floating-point LP solver. Since such solvers do not typically provide a bound on the accuracy of their solutions (Held et al., 2012), $(\pi^{\text{float}}, \rho^{\text{float}})$ may be affected by numerical errors (it may be infeasible, suboptimal, or both). This may not be an issue *per se*. Indeed, a valid dual bound could be recovered from an infeasible dual solution by, e.g., manipulating it as done for set covering problems by Farley (1990) as well as from a suboptimal dual solution by, e.g., including an additive corrective term in the bound calculations (which should, though, be done without numerical errors) as done with the *Lagrangian bound* (Lübbecke and Desrosiers, 2005). These procedures, though, require the calculations carried out by the pricer on $(\pi^{\text{float}}, \rho^{\text{float}})$ to be numerically exact.

Due to the floating-point nature of $(\pi^{\text{float}}, \rho^{\text{float}})$, though, cancellation errors may be made in the pricing algorithm even if the dual solution were perfectly accurate whenever the precision that is available to the pricer is insufficient to carry out the necessary arithmetic operations on $(\pi^{\text{float}}, \rho^{\text{float}})$ without errors. As a consequence, the pricing algorithm may attest the dual

feasibility of $(\boldsymbol{\pi}^{\text{float}}, \boldsymbol{\rho}^{\text{float}})$ (thus declaring $\bar{c}_S(\boldsymbol{\pi}^{\text{float}}, \boldsymbol{\rho}^{\text{float}}, 1) \geq 0$ for all $S \in \widehat{\mathcal{P}}$) even if the solution is not dual feasible and $z^{\text{float}} > z(\text{MP})$ or deem $(\boldsymbol{\pi}^{\text{float}}, \boldsymbol{\rho}^{\text{float}})$ infeasible even if it is not. In the first case, the risk is of computing the invalid lower bound $\lceil z^{\text{float}} \rceil > z(\text{BPP})$, while, in the second case, the column-and-row-generation algorithm may enter an infinite loop.

To overcome these issues so as to be able to work with dual variables that can be safely manipulated by the pricing algorithm without incurring cancellation errors, we extend the technique proposed by Held et al. (2012) to the case where the RMP features nonnegative as well as nonpositive dual variables. We approximate the (possibly inaccurate) floating-point dual solution $(\boldsymbol{\pi}^{\text{float}}, \boldsymbol{\rho}^{\text{float}})$ by introducing the following *scaled integer* dual solution $(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}})$:

$$\pi_j^{\text{int}} = \lfloor K \pi_j^{\text{float}} \rfloor, \quad j \in \overline{N}, \quad \text{and} \quad \rho_T^{\text{int}} = \lfloor K \rho_T^{\text{float}} \rfloor, \quad T \in \overline{\mathcal{T}},$$

where $K > 0$ is a positive scaling factor and the following *diminished* dual solution $(\boldsymbol{\pi}^{\text{dim}}, \boldsymbol{\rho}^{\text{dim}})$:

$$\pi_j^{\text{dim}} = \frac{1}{K} \pi_j^{\text{int}}, \quad j \in \overline{N}, \quad \text{and} \quad \rho_T^{\text{dim}} = \frac{1}{K} \rho_T^{\text{int}}, \quad T \in \overline{\mathcal{T}}.$$

The parameter K affects the numerical precision of the column-and-row-generation algorithm. In Section 6.6, we describe how to select a suitable value for K and how varying K impacts the algorithm's efficiency. If K is chosen as a power of 10, i.e., $K = 10^k$ for some $k \in \mathbb{N}$, then $\boldsymbol{\pi}^{\text{dim}} = \text{trunc}(\boldsymbol{\pi}^{\text{float}}, k)$ due to $\boldsymbol{\pi}^{\text{float}} \geq \mathbf{0}$, where $\text{trunc}(x, k)$ is the truncation of the floating-point number x to the k -th decimal figure. Differently, $\boldsymbol{\rho}^{\text{dim}}$ does *not* coincide with $\text{trunc}(\boldsymbol{\rho}^{\text{float}}, k)$ as, due to $\boldsymbol{\rho}^{\text{float}} \leq \mathbf{0}$, the latter is equal to $\lceil \frac{1}{K} \boldsymbol{\rho}^{\text{float}} \rceil$ whereas $\boldsymbol{\rho}^{\text{dim}}$ is defined as $\lfloor \frac{1}{K} \boldsymbol{\rho}^{\text{float}} \rfloor$. Thus, $\boldsymbol{\pi}^{\text{dim}} = \text{trunc}(\boldsymbol{\pi}^{\text{float}}, k) \leq \boldsymbol{\pi}^{\text{float}}$ whereas $\boldsymbol{\rho}^{\text{dim}} \leq \boldsymbol{\rho}^{\text{float}} \leq \text{trunc}(\boldsymbol{\rho}^{\text{float}}, k)$. By construction, if $(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}})$ satisfies

$$\sum_{j \in S} \pi_j^{\text{int}} + \sum_{T \in \overline{\mathcal{T}}(S)} \rho_T^{\text{int}} \leq K, \quad \forall S \in \widehat{\mathcal{P}},$$

then $(\boldsymbol{\pi}^{\text{dim}}, \boldsymbol{\rho}^{\text{dim}})$ satisfies the dual Constraints (2b). Thus, it is dual feasible for the RMP and $z^{\text{dim}} = \sum_{j \in \overline{N}} \pi_j^{\text{dim}} + \sum_{T \in \overline{\mathcal{T}}} \rho_T^{\text{dim}}$ is a valid lower bound on $z(\text{MP})$ (i.e., $z^{\text{dim}} \leq z(\text{MP})$). Similar considerations apply when $\boldsymbol{\pi}^{\text{float}}$ and $\boldsymbol{\rho}^{\text{float}}$ have a binary representation and, as done in our computational experiments, K is chosen so as to satisfy $K = 2^k$ for some $k \in \mathbb{N}$.

After computing the scaled integer dual vectors $(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}})$, we use them for solving the pricing problem in order to compute one or more patterns by minimizing, rather than the *floating-point reduced cost* $\bar{c}_S(\boldsymbol{\pi}^{\text{float}}, \boldsymbol{\rho}^{\text{float}}, 1)$ (doing so may lead to incorrect calculations due to the fixed-point nature of the pricer), the *scaled integer reduced cost* $\bar{c}_S(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}}, K)$. Notice that $\bar{c}_S(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}}, K)$ is equal to the *diminished reduced cost* $\bar{c}_S(\boldsymbol{\pi}^{\text{dim}}, \boldsymbol{\rho}^{\text{dim}}, 1)$ multiplied by K . This way, the pricing algorithm works solely with fixed-point (integer) numbers. Assuming that $(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}})$ and the result of any intermediate calculations that the pricer carries out can be represented by the integer data type that is used in its implementation, cancellation errors are completely avoided and the pricer always produces a numerically-exact solution.

From a convergence perspective, the definition of $(\boldsymbol{\pi}^{\text{dim}}, \boldsymbol{\rho}^{\text{dim}})$ is motivated by the following proposition:

Proposition 3.1. *Consider a column-and-row-generation method based on a floating-point LP solver whose calculations are guaranteed to be in an (absolute) additive error no larger than ϵ_0 , in which at each iteration the pricing problem is solved with the integer dual variables $(\pi^{\text{int}}, \rho^{\text{int}})$ without numerical errors, and where the column-and-row-generation method stops whenever $\min_{S \in \widehat{\mathcal{P}}} \{\bar{c}_S(\pi^{\text{dim}}, \rho^{\text{dim}}, 1)\} \geq -\epsilon_0$. Then, the column-and-row-generation algorithm terminates after finitely many iterations.*

Proof. By construction, we have $\pi_j^{\text{float}} \geq \pi_j^{\text{dim}}$ for all $j \in \bar{N}$ and $\rho_T^{\text{float}} \geq \rho_T^{\text{dim}}$ for all $T \in \bar{\mathcal{T}}$. Thus,

$$\begin{aligned} \frac{1}{K} \bar{c}_S(\pi^{\text{int}}, \rho^{\text{int}}, K) &= \bar{c}_S(\pi^{\text{dim}}, \rho^{\text{dim}}, 1) = 1 - \sum_{j \in S} \pi_j^{\text{dim}} - \sum_{T \in \bar{\mathcal{T}}(S)} \rho_T^{\text{dim}} \geq \\ &1 - \sum_{j \in S} \pi_j^{\text{float}} - \sum_{T \in \bar{\mathcal{T}}(S)} \rho_T^{\text{float}} = \bar{c}_S(\pi^{\text{float}}, \rho^{\text{float}}, 1) \end{aligned} \quad (4)$$

holds for each $S \in \widehat{\mathcal{P}}$. Given a dual solution $(\pi^{\text{float}}, \rho^{\text{float}})$ and the corresponding diminished dual solution $(\pi^{\text{dim}}, \rho^{\text{dim}})$, for each S belonging to the set of patterns $\bar{\mathcal{P}} \subseteq \widehat{\mathcal{P}}$ already contained in the RMP we have $\bar{c}_S(\pi^{\text{dim}}, \rho^{\text{dim}}, 1) \geq \bar{c}_S(\pi^{\text{float}}, \rho^{\text{float}}, 1) \geq -\epsilon_0$ and, thus, already generated patterns cannot be generated once again. By assumption, if $(\pi^{\text{dim}}, \rho^{\text{dim}})$ violates the dual constraint associated with a pattern S of diminished reduced cost $\bar{c}_S(\pi^{\text{dim}}, \rho^{\text{dim}}, 1) < 0$, then $\bar{c}_S(\pi^{\text{dim}}, \rho^{\text{dim}}, 1) < -\epsilon_0$ holds. Due to (4), this implies $\bar{c}_S(\pi^{\text{float}}, \rho^{\text{float}}, 1) < -\epsilon_0$. Therefore, such a constraint is violated by the floating-point solution $(\pi^{\text{float}}, \rho^{\text{float}})$ returned by the floating-point LP solver. Since its reduced cost is strictly smaller than $-\epsilon_0$, when the column corresponding to pattern S is added to the RMP, $(\pi^{\text{float}}, \rho^{\text{float}})$ becomes numerically infeasible for the floating-point solver. This guarantees that, if the reduced costs calculated by the LP solver are guaranteed to be in an absolute error no larger than ϵ_0 , the column-and-row-generation method will not enter an infinite loop. Hence, since $\widehat{\mathcal{P}}$ contains only finitely many patterns, the column-and-row-generation method terminates after finitely many iterations. $\square$

$\square$

Proposition 3.1 assumes that the LP solver is guaranteed to carry out its calculations in an (absolute) error no larger than its tolerance ϵ_0 . This assumption, though, is not guaranteed to hold with state-of-the-art floating-point (inexact) LP solvers. Such solvers allow the user to set the value of tolerance parameters such as e.g., `CXPARAM_Simplex_Tolerances_Optimality` and `OptimalityTol` for, respectively, `Cplex` and `Gurobi`. Such a parameter, though, is used as cut-off value within a termination condition whose computation may itself be subject to numerical errors. Therefore, such a parameter does not typically coincide with a bound on the cancellation error the floating-point solver may be in (this is exacerbated by, among others, the adoption of pre-solve and scaling techniques). Thus, the condition that $\bar{c}_S(\pi^{\text{dim}}, \rho^{\text{dim}}, 1) < -\epsilon_0$ implies $\bar{c}_S(\pi^{\text{float}}, \rho^{\text{float}}, 1) < -\epsilon_0$ may not necessarily hold in practice. If violated, the column-and-row-generation method may enter an infinite loop where the floating-point dual solution $(\pi^{\text{float}}, \rho^{\text{float}})$ remains unchanged (due to remaining feasible) regardless of the tolerance that is selected for the solver. We must note, however, that we have never observed such a phenomenon

in our numerical experiments.

To avoid infinite loops and guarantee convergence (albeit to a weaker bound) under a more general setting, whenever one or more patterns S with $\bar{c}_S(\boldsymbol{\pi}^{\dim}, \boldsymbol{\rho}^{\dim}, 1) < -\epsilon_0$ are generated, one can check whether at least one of such patterns enters the basis in the next RMP solution and, if not, terminate the column-and-row-generation algorithm and set ϵ_0 to the smallest value taken by $\bar{c}_S(\boldsymbol{\pi}^{\dim}, \boldsymbol{\rho}^{\dim}, 1)$ across the new patterns. We remark that, in the context of the two-phase column-and-row-generation method that we will introduce in the following, if this situation were to occur, the algorithm would naturally enter phase II where the infinite-precision LP solver is used, thus achieving convergence.

In order for the column-and-row-generation algorithm to converge, K must therefore be chosen in accordance with the data type used for storing integer values in one's implementation of the pricing algorithm in such a way that integer overflows are avoided (see Section 6.6 for the related details) as well as with the precision of the floating-point solver. Note that this also applies to other contexts such as the set-covering formulation of Held et al. (2012).

The column-and-row-generation algorithm is guaranteed to terminate when either working (as assumed in Proposition 3.1) with a floating-point LP solver that is guaranteed to be in an absolute error that is sufficiently small for one's choice of K or when ϵ_0 is chosen as explained before when such an assumption is not guaranteed to hold. However, working with the diminished dual solution $(\boldsymbol{\pi}^{\dim}, \boldsymbol{\rho}^{\dim})$ rather than with $(\boldsymbol{\pi}^{\text{float}}, \boldsymbol{\rho}^{\text{float}})$ can lead to a bound $z^{\dim}$ that is weaker than $z(\text{MP})$. The difference between the two can be bounded according to the following proposition:

Proposition 3.2. *Let $(\boldsymbol{\pi}^{\text{float}}, \boldsymbol{\rho}^{\text{float}})$ be an (error-free) optimal dual feasible solution to the MP of value $z(\text{MP})$ and let $(\boldsymbol{\pi}^{\dim}, \boldsymbol{\rho}^{\dim})$ be the associated diminished solution of value $z^{\dim}$. Then, $z(\text{MP}) - \frac{|\bar{N}| + |\bar{\mathcal{T}}|}{K} \leq z^{\dim} \leq z(\text{MP})$.*

Proof. For any $\alpha, \beta \in \mathbb{R}$ such that $\beta = \lfloor K\alpha \rfloor$ we have $\alpha - \frac{1}{K}\beta \leq \frac{1}{K}$; hence, we deduce $0 \leq \pi_j^{\text{float}} - \pi_j^{\dim} \leq \frac{1}{K}$ for all $j \in \bar{N}$ and $0 \leq \rho_T^{\text{float}} - \rho_T^{\dim} \leq \frac{1}{K}$ for all $T \in \bar{\mathcal{T}}$. Since

$$z^{\text{float}} - z^{\dim} = \sum_{j \in \bar{N}} (\pi_j^{\text{float}} - \pi_j^{\dim}) + \sum_{T \in \bar{\mathcal{T}}} (\rho_T^{\text{float}} - \rho_T^{\dim}),$$

we deduce

$$0 \leq z^{\text{float}} - z^{\dim} \leq \frac{|\bar{N}| + |\bar{\mathcal{T}}|}{K}.$$

The claim follows since $z^{\text{float}} = z(\text{MP})$. □

The proposition shows that $\frac{1}{K}$ should be as small as possible as, if $\frac{1}{K} \geq \epsilon_0$, the final dual bound obtained when the scaling procedure is adopted would be unnecessarily weak.

To achieve a higher numerical precision than $\frac{1}{\epsilon_0}$, one could, in principle, adopt a rational (infinite-precision) LP solver instead of a floating-point one at each iteration of the column-and-row-generation method. In such a case, K would be selected purely based on the fixed-point precision of the pricing algorithm. As known, though, infinite-precision LP solvers suffer from

significantly higher running times than their floating-point counterparts, and their adoption at each stage of the column-and-row-generation method is likely to be impractical (which is indeed confirmed by our preliminary computational experiments).

For this reason, we introduce a two-phase column-and-row-generation algorithm to efficiently handle the dual bound calculations with a value of K satisfying $\frac{1}{K} \ll \epsilon_0$. In the first phase, we rely on the more efficient but inaccurate floating-point LP solver. In the second phase, we switch to the rational solver and use it until the MP is solved. This way, we limit the number of calls to the rational solver and achieve a good efficiency as well as a high numerical accuracy.

3.2 Two-phase column generation algorithm

We now describe in more detail the two-phase column-and-row-generation algorithm we propose to solve the MP at each node of the enumeration tree. Before doing so, we introduce dual bounds and fathoming rules that we leverage to improve the overall computational efficiency of the method.

3.2.1 Lower bounds and fathoming rules

To achieve, if possible, an early termination of the column-and-row-generation algorithm, we rely on an extension to the case with SR3 constraints of the classical bound that holds for set covering problems that is due to Farley (1990) and also adopt two (classical) fathoming rules (or termination criteria).

The extension of Farley's bound is described by the following proposition.

Proposition 3.3. *Let $(\pi^{\dim}, \rho^{\dim})$ be a (not necessarily feasible nor optimal) diminished dual solution of value $z^{\dim}$ to the RMP at an iteration of the column-and-row-generation procedure. Let $\bar{c}_{\min} = \min_{S \in \widehat{\mathcal{P}}} \{\bar{c}_S(\pi^{\dim}, \rho^{\dim}, 1)\}$. If $\bar{c}_{\min} < 0$, the value*

$$LB_F = \frac{z^{\dim}}{1 - \bar{c}_{\min}}$$

is a valid lower bound on the optimal solution value $z(MP)$ of the MP.

Proof. By assumption, we have

$$\bar{c}_S(\pi^{\dim}, \rho^{\dim}, 1) = 1 - \sum_{j \in S} \pi_j^{\dim} - \sum_{T \in \overline{\mathcal{T}}(S)} \rho_T^{\dim} \geq \bar{c}_{\min}, \quad \forall S \in \widehat{\mathcal{P}},$$

or, equivalently,

$$\sum_{j \in S} \pi_j^{\dim} + \sum_{T \in \overline{\mathcal{T}}(S)} \rho_T^{\dim} \leq 1 - \bar{c}_{\min}, \quad \forall S \in \widehat{\mathcal{P}}.$$

Since $1 - \bar{c}_{\min} > 0$, we deduce

$$\sum_{j \in S} \frac{\pi_j^{\dim}}{1 - \bar{c}_{\min}} + \sum_{T \in \overline{\mathcal{T}}(S)} \frac{\rho_T^{\dim}}{1 - \bar{c}_{\min}} \leq 1, \quad \forall S \in \widehat{\mathcal{P}}.$$

This is to say that $\bar{c}_S(\frac{\pi^{\dim}}{1-\bar{c}_{\min}}, \frac{\rho^{\dim}}{1-\bar{c}_{\min}}, 1) \geq 0$ holds for all $S \in \widehat{\mathcal{P}}$. Therefore, the dual solution $(\frac{\pi^{\dim}}{1-\bar{c}_{\min}}, \frac{\rho^{\dim}}{1-\bar{c}_{\min}})$ of value $\frac{z^{\dim}}{1-\bar{c}_{\min}}$ is a feasible solution to the dual of the MP and, thus, $\frac{z^{\dim}}{1-\bar{c}_{\min}}$ is a valid lower bound on $z(\text{MP})$. $\square$ $\square$

Interestingly, LB_F turns out to be tighter than the classical lower bound on $z(\text{MP})$ known in the literature as the *Lagrangian bound* (see [Lübbecke and Desrosiers, 2005](#)), which reads:

$$LB_L = z^{\dim} + UB \bar{c}_{\min},$$

where UB is an upper bound on the optimal solution value of the MP. The validity of LB_L follows directly from the theory of linear programming as the value of $z^{\dim}$ cannot be reduced by more than UB times the smallest reduced cost $\bar{c}_{\min}$. Thus, we have $z^{\dim} + UB \bar{c}_{\min} \leq z(\text{MP}) \leq z^{\dim}$.

The following proposition holds.

Proposition 3.4. *LB_F dominates LB_L , i.e., $LB_F \geq LB_L$.*

Proof. We aim to show that

$$LB_L = z^{\dim} + UB \bar{c}_{\min} \leq \frac{z^{\dim}}{1 - \bar{c}_{\min}} = LB_F.$$

Since $\bar{c}_{\min} < 0$, by dividing both sides by $\bar{c}_{\min} < 0$, the previous inequality is shown to be equivalent to the following one:

$$\frac{z^{\dim}}{\bar{c}_{\min}} + UB \geq \frac{z^{\dim}}{\bar{c}_{\min}(1 - \bar{c}_{\min})}.$$

Grouping by $z^{\dim}$, the previous inequality can be rewritten as follows:

$$UB \geq z^{\dim} \left(\frac{1}{\bar{c}_{\min}(1 - \bar{c}_{\min})} - \frac{1}{\bar{c}_{\min}} \right) = \frac{z^{\dim}}{1 - \bar{c}_{\min}} = LB_F.$$

As the latter inequality is always satisfied thanks to Proposition 3.3, the claim follows. $\square$ $\square$

We remark that this result holds even for set covering problems where LB_F is the classical bound due to Farley. To the best of our knowledge, this result is new.

In our algorithm, LB_F is combined with the following two fathoming rules. The first one leverages the value of the incumbent upper bound UB to fathom (when the latter is optimal) a node of the enumeration tree:

Fathoming Rule 3.5. If $\lceil LB_F \rceil = UB$, the subproblem associated with the current node of the enumeration tree is fathomed by the upper bound UB .

The second one leverages the value of the lower bound $\lceil z^{\dim} \rceil$ to achieve (when the latter is optimal) an early-termination of the column-and-row generation procedure at a node of the enumeration tree:

Fathoming Rule 3.6. Let $z^{\dim}$ be the solution value of the RMP at an iteration of the column-and-row-generation method. If $\lceil LB_F \rceil = \lceil z^{\dim} \rceil$, the column-and-row-generation procedure is halted and a valid lower bound $\lceil LB_F \rceil$ at the node is determined.

3.2.2 Alternative derivation of the Lagrangian bound

We provide, here, an alternative derivation of the Lagrangian bound which, to the best of our knowledge, is new and could be of independent interest. Consider an extension of the SC formulation obtained by adding to it the redundant constraint $\sum_{S \in \overline{\mathcal{P}}} \xi_S \leq UB$, where $UB \leq n$ is an upper bound on the number of bins featured in an optimal solution. The dual of such a problem reads:

$$\max_{\pi \geq 0, \rho \leq 0, \gamma \leq 0} \left\{ \sum_{j \in \overline{N}} \pi_j + \sum_{T \in \overline{\mathcal{T}}} \rho_T + UB\gamma : \sum_{j \in S} \pi_j + \sum_{T \in \overline{\mathcal{T}}(S)} \rho_T + \gamma \leq 1, S \in \overline{\mathcal{P}} \right\}.$$

Given a floating-point solution $(\pi^{\text{float}}, \rho^{\text{float}})$, let $\gamma^{\text{float}} = \min_{S \in \overline{\mathcal{P}}} \{1 - \sum_{j \in S} \pi_j^{\text{float}} - \sum_{T \in \overline{\mathcal{T}}(S)} \rho_T^{\text{float}}\}$, where we assume that γ^{float} is computed with infinite precision. If $\gamma^{\text{float}} \geq 0$, $(\pi^{\text{float}}, \rho^{\text{float}})$ is dual feasible (with infinite precision). If not, the invalid dual bound $\sum_{j \in \overline{N}} \pi_j^{\text{float}} + \sum_{T \in \overline{\mathcal{T}}} \rho_T^{\text{float}}$ can be corrected by adding to it the (negative) term $UB \gamma^{\text{float}}$. As, by construction, $\gamma^{\text{float}} = \bar{c}_{\min}$, such an updated bound coincides with the Lagrangian bound.

3.3 Outline of the two-phase column-generation algorithm

The algorithm we use to solve the MP at a node of the enumeration tree is summarized by Algorithm 1. It takes as input the initial numerical tolerance ϵ_0 associated with the LP solver, the value of K , the sets $\overline{\mathcal{P}}$ and $\overline{\mathcal{T}}$ of, respectively, the patterns and SR3 constraints associated with the current RMP, and the incumbent value UB . It returns the updated sets $\overline{\mathcal{P}}$ and $\overline{\mathcal{T}}$, the value of the lower bound $z^{\dim}$ and (these values will be used for the pattern enumeration method described below) the value of z^{int} , the integer scaled dual vector $(\pi^{\text{int}}, \rho^{\text{int}})$, and the diminished dual vector $(\pi^{\dim}, \rho^{\dim})$. If $z^{\dim} = +\infty$, the subproblem associated with the BPC node is fathomed by Fathoming Rule 3.5.

The algorithm starts by setting ϵ to the numerical tolerance ϵ_0 of the floating-point LP solver (phase I). The variable ‘optimality’ takes value ‘true’ if the current lower bound is proven to be optimal and ‘false’ otherwise, whereas the variable ‘fathomed’ takes value ‘true’ if the current subproblem is fathomed and ‘false’ otherwise. At each iteration of the main loop (line 2), the algorithm solves the RMP using the LP solver with tolerance ϵ and computes the diminished and scaled integer dual vectors $(\pi^{\dim}, \rho^{\dim})$ and $(\pi^{\text{int}}, \rho^{\text{int}})$ (lines 3-4). Then, the pricing problem is solved using $(\pi^{\text{int}}, \rho^{\text{int}})$ followed by the computation of the scaled integer reduced cost $\bar{c}_{\min} = \min_{S \in \overline{\mathcal{P}}} \{\bar{c}_S(\pi^{\text{int}}, \rho^{\text{int}}, K)\}$ associated with the set of generated patterns $\mathcal{P}'$ (lines 5-6). Fathoming Rules 3.5 and 3.6 are then tested (lines 7-8); if Fathoming Rule 3.6 applies, the value of the lower bound $z^{\dim}$ is set to LB_F . The sign of $\bar{c}_{\min}$ is then checked to verify the convergence of the algorithm (line 9). If $\bar{c}_{\min} \geq 0$, SR3 inequalities are separated (line 10). If none are found, optimality is proven (line 12); otherwise, the corresponding set of

Algorithm 1: Two-phase column generation algorithm

Input: $\epsilon_0, K, \overline{\mathcal{P}}$ and $\overline{\mathcal{T}}, UB$ **Output:** $\overline{\mathcal{P}}, \overline{\mathcal{T}}, z^{\text{int}}, z^{\text{dim}}, (\pi^{\text{int}}, \rho^{\text{int}}), (\pi^{\text{dim}}, \rho^{\text{dim}})$ **begin**

// Initialization (phase I starts)

1 $\epsilon \leftarrow \epsilon_0$, optimality $\leftarrow$ false, fathomed $\leftarrow$ false;

// Main loop

2 **while** optimality = false **and** fathomed = false **do**

// Solve the RMP

3 Solve the RMP using solver $LP(\epsilon)$. Let $(\pi^{\text{float}}, \rho^{\text{float}})$ be the RMP dual solution;

// Compute scaled integer and diminished dual solutions

4 Compute the scaled integer and the diminished dual solutions $(\pi^{\text{int}}, \rho^{\text{int}})$ and $(\pi^{\text{dim}}, \rho^{\text{dim}})$;

// Solve the pricing problem

5 Compute $S' \in \arg \min_{S \in \widehat{\mathcal{P}}} \{\bar{c}_S(\pi^{\text{int}}, \rho^{\text{int}}, K)\}$ and let $\mathcal{P}' \subseteq \{S \in \widehat{\mathcal{P}} : \bar{c}_S(\pi^{\text{int}}, \rho^{\text{int}}, K) < 0\}$ (see §4.1); // Computation of $\bar{c}_{\min}$ 6 $\bar{c}_{\min} \leftarrow \bar{c}_{S'}(\pi^{\text{dim}}, \rho^{\text{dim}}, 1)$;

// Apply Fathoming rule 3.5

7 **if** $[LB_F] = UB$ **then** fathomed $\leftarrow$ true, $z^{\text{dim}} \leftarrow +\infty$; **break** ;

// Apply Fathoming rule 3.6

8 **if** $[LB_F] = \lceil z^{\text{dim}} \rceil$ **then** optimality $\leftarrow$ true, $z^{\text{dim}} \leftarrow LB_F$; **break** ;

// Check convergence

9 **if** $\bar{c}_{\min} \geq 0$ **then**

// SR3 separation

10 Separate SR3 by complete enumeration. Let $\mathcal{T}' \subseteq \mathcal{T} \setminus \overline{\mathcal{T}}$ be the set of generated triplets;11 **if** $|\mathcal{T}'| = 0$ **then**12 | optimality $\leftarrow$ true;13 **else**

// Update the set of SR3

14 | $\overline{\mathcal{T}} \leftarrow \overline{\mathcal{T}} \cup \mathcal{T}'$;15 **end**16 **else**

// Update the set of patterns

17 $\overline{\mathcal{P}} \leftarrow \overline{\mathcal{P}} \cup \mathcal{P}' \cup \{S'\}$;

// Check LP numerical precision

18 **if** $\bar{c}_{\min} \geq -\epsilon$ **then**

// Set precision to infinite-precision (phase II starts)

19 | $\epsilon \leftarrow 0$;20 **end**21 **end**22 **end**23 **end**

SR3 inequalities $\overline{\mathcal{T}}$ is updated (line 14). If $\bar{c}_{\min} < 0$, the set of patterns $\overline{\mathcal{P}}$ is updated (line 17). The algorithm then checks if the current tolerance ϵ of the LP solver is still sufficient to include (one or more) of the newly generated patterns in $\mathcal{P}'$ in the basis at the next iteration (line 18). If this is not the case, the tolerance parameter ϵ is set to 0, in which case the rational (infinite-precision) solver is adopted from the next main iteration until convergence is established (phase II).

Since, due to working with fixed-point integer numbers, the maximum nonpositive scaled integer reduced cost $\bar{c}_S(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}}, K)$ that can be computed by the pricing algorithm is equal to -1 , the maximum nonpositive diminished reduced cost $\bar{c}_S(\boldsymbol{\pi}^{\text{dim}}, \boldsymbol{\rho}^{\text{dim}}, 1)$ that can be computed by the pricing algorithm is equal to $-\frac{1}{K}$. If K is chosen so as to satisfy $K = \frac{1}{\epsilon_0}$ (where $\epsilon_0 > 0$ is the tolerance of the LP solver), then the maximum negative diminished reduced cost value computed by the pricing algorithm is no larger than $-\frac{1}{\epsilon_0}$ and, therefore, Algorithm 1 works within the tolerance of the LP solver. Dual bounds with an extended numerical precision higher than $\frac{1}{\epsilon_0}$ are obtained for any $K > \frac{1}{\epsilon_0}$.

4 Pricing algorithm and pattern enumeration

In this section, we provide a description of the exact integer (fixed-point) pricing algorithm we propose and of the pattern enumeration method, together with an illustration of how we solve the reduced SC problem to optimality in a numerically-exact way thanks to a simplified version of our BPC algorithm.

4.1 Mathematical formulation of the pricing problem and pricing algorithm

As described in Section 2, the pricing problem corresponds to an instance of the KP-T-C and can be mathematically formulated as follows. We recall that we are given a set $\overline{N} = \{1, 2, \dots, \bar{n}\}$ of $\bar{n}$ items, each having an integer weight $\bar{w}_j$, $j \in \overline{N}$. We are also given a conflict graph $G = (\overline{N}, E)$, where E is a set of edges such that $\{i, j\} \in E$ when i and j are in *conflict*.

Let x_j be a binary variable equal to 1 if item $j \in \overline{N}$ is selected, 0 otherwise. Further, let z_T be a binary variable equal to 1 if and only if the dual contribution of $T \in \overline{\mathcal{T}}$ must be subtracted from the reduced cost of an optimal pattern. The KP-T-C can be formulated as follows:

$$\min \quad b - \sum_{j \in \overline{N}} \pi_j x_j - \sum_{T \in \overline{\mathcal{T}}} \rho_T z_T \quad (5a)$$

$$\text{s.t.} \quad \sum_{j \in \overline{N}} \bar{w}_j x_j \leq W \quad (5b)$$

$$x_{j_1} + x_{j_2} \leq 1 \quad \{j_1, j_2\} \in E \quad (5c)$$

$$x_{j_1} + x_{j_2} \leq z_T + 1 \quad T \in \overline{\mathcal{T}}, j_1, j_2 \in T, j_1 \neq j_2 \quad (5d)$$

$$x_j \in \{0, 1\} \quad j \in \overline{N} \quad (5e)$$

$$z_T \in \{0, 1\} \quad T \in \overline{\mathcal{T}}. \quad (5f)$$

The objective function in (5a) corresponds to the reduced cost (see Equation (3)) to be minimized.

Constraint (5b) imposes that the knapsack capacity be not exceeded. Constraints (5c) avoid that pairs of items which are in conflict be packed together in a solution. Constraints (5d) guarantee that the dual contributions of the set of inequalities in $\overline{\mathcal{T}}$ are accounted for in z_T by imposing $z_T = 1$ for each $T \in \overline{\mathcal{T}}, j_1, j_2 \in T, j_1 \neq j_2$ whenever $x_{j_1} = x_{j_2} = 1$. Notice that w.l.o.g. the integrality restrictions on the z variables can be lifted.

Below, we describe an algorithm for the solution of the KP-T-C based on Dynamic Programming (DP). According to a set of preliminary experiments, our DP algorithm outperforms the direct solution of Formulation (5) using state-of-the-art MILP solvers. Moreover, our DP algorithm, as required by Algorithm 1, works in exact integer (fixed-point) precision, a property that is not guaranteed by state-of-the-art MILP solvers.

The pricing algorithm follows the scheme of a classical DP algorithm for the KP (see Martello and Toth, 1990) where $|\overline{N}|$ stages are considered and each stage $j \in \overline{N}$ is associated with a subset of the set of feasible patterns $\widehat{\mathcal{P}}$ involving the first j items. The algorithm takes as input an integer dual solution (π, ρ) and two user-defined parameters $\hat{z} \in \mathbb{Z}$ and $\Delta \in \mathbb{N}$. If $\hat{z} < 0$, the algorithm produces as output the most negative reduced cost pattern together with up to $\Delta - 1$ additional negative reduced cost patterns (if any). If $\hat{z} \geq 0$, pattern enumeration is performed (see Subsection 4.2 for a detailed description) and the algorithm generates at most Δ patterns with a reduced cost no larger than $\hat{z}$.

We call a tuple $L = (j, X, \omega, \bar{c}, Y, \mathcal{R})$ representing a *partial* pattern a *label*. In it: j is the last item featured in the label; $X \subseteq \{1, \dots, j\}$ is the set of items contained in the partial pattern; ω is the total weight of X , i.e., $\omega = \sum_{h \in X} \bar{w}_h$; $\bar{c}$ is the reduced cost of the partial pattern; Y is the set of items in $\{j+1, \dots, \bar{n}\}$ that can be individually added to the set X while keeping the pattern feasible and $\mathcal{R}$ is the set of $|\overline{\mathcal{T}}|$ binary resources associated with the triplets in the set $\overline{\mathcal{T}}$, namely, $\mathcal{R} = \{\sigma_1, \sigma_2, \dots, \sigma_{|\overline{\mathcal{T}}|}\}$ and $\sigma : \widehat{\mathcal{P}} \rightarrow \{0, 1\}$ for all $\sigma \in \mathcal{R}$. With a slight abuse of notation, we refer to the attributes $j, X, \omega, \bar{c}, Y$, and $\mathcal{R}$ of a label L by $j(L), X(L), \omega(L), \bar{c}(L), Y(L)$, and $\mathcal{R}(L)$.

The algorithm starts from the initial label $L_0 = (0, \emptyset, 0, 1, \overline{N}, \{0\}_{\sigma \in \mathcal{R}})$. A label $L = (j, X, \omega, \bar{c}, Y, \mathcal{R})$ represents a complete pattern consisting of the set of items in X of reduced cost equal to $\bar{c}$ if $Y = \emptyset$. If $Y \neq \emptyset$, let $j^* = \arg \min\{h : h \in Y\}$ be the item of minimum index in Y . Given a label L , a *label extension* rule is used to create two new labels L_1 and L_2 by packing j^* in the former and discarding it in the latter. For a given label L , let $\widehat{\mathcal{P}}(L) \subseteq \widehat{\mathcal{P}}$ be the set of all feasible patterns with items in $Y(L)$ that can be used to complete the partial pattern $X(L)$, i.e., $\widehat{\mathcal{P}}(L) = \{S' \in \overline{N} \mid S' = S \setminus X(L) \text{ for some } S \in \widehat{\mathcal{P}}, X(L) \subset S'\}$. Given a label $L = (j, X, \omega, \bar{c}, Y, \mathcal{R})$ and a pattern $S \in \widehat{\mathcal{P}}(L)$, we denote by $(X(L), S)$ the pattern $(X(L) \cup S) \in \widehat{\mathcal{P}}$ obtained by joining the partial pattern $X(L)$ with its completion S .

The algorithm relies on fathoming and dominance rules to reduce the number of labels it works with and accelerate its execution. The following property holds:

Property 4.1. *Given a label $L = (j, X, \omega, \bar{c}, Y, \mathcal{R})$, let $lb(L)$ be a lower bound on the reduced cost of any pattern $(X(L), S) \in \widehat{\mathcal{P}}$ that can be obtained by completing the partial pattern $X(L)$*

with some $S \in \widehat{\mathcal{P}}(L)$. If $lb(L) \geq \hat{z}$, then the label L cannot lead to any pattern of reduced cost smaller than $\hat{z}$ and, therefore, it can be discarded.

Let $KP(B, \alpha, \beta, \gamma)$ be an instance of the KP (in minimization version) asking for a minimum-profit subset of items of $B = \{1, 2, \dots, |B|\}$ which fit into a bin of capacity $\alpha \in \mathbb{Z}_+$ with profits $\beta \in \mathbb{R}_-^{|B|}$ and weights $\gamma \in \mathbb{Z}_+^{|B|}$. Let $z(KP(B, \alpha, \beta, \gamma))$ be its optimal solution value. We define the following fathoming rule (also used by Wei et al. (2020)).

DP Fathoming Rule 4.2. Given a label $L = (j, X, \omega, \bar{c}, Y, \mathcal{R})$, the value $lb_1(L) = \bar{c}(L) + z(KP(\{j+1, \dots, n\}, c - \omega(L), \beta, \gamma))$ with $\beta = (-\pi_{j+1}, \dots, -\pi_n)$ and $\gamma = (\bar{w}_{j+1}, \dots, \bar{w}_n)$ provides a valid lower bound on the reduced cost of any pattern $(X(L), S) \in \widehat{\mathcal{P}}, S \in \widehat{\mathcal{P}}(L)$.

Let $z(KP_{LP}(B, \alpha, \beta, \gamma))$ be the optimal solution value of the LP relaxation of the aforementioned KP problem. We introduce the following new fathoming rule:

DP Fathoming Rule 4.3. Given a label $L = (j, X, \omega, \bar{c}, Y, \mathcal{R})$, the value $lb_2(L) = \bar{c}(L) + z(KP_{LP}(B, c - \omega(L), \beta, \gamma))$ provides a valid lower bound on the reduced cost of any pattern $(X(L), S) \in \widehat{\mathcal{P}}, S \in \widehat{\mathcal{P}}(L)$, where:

- B is equal to $Y(L)$ (we assume that the indices in B are defined consecutively);
- $\beta = (-\pi_{\nu(1)}, \dots, -\pi_{\nu(|Y(L)|)})$ and $\gamma = (\bar{w}_{\nu(1)}, \dots, \bar{w}_{\nu(|Y(L)|)})$, where, for each $h \in B$, $\nu(h)$ is the original index of the item of index h in $Y(L)$.

Below, we show how the lower-bounding functions $lb_1(\cdot)$ and $lb_2(\cdot)$ can be efficiently computed.

It is worth noting that, differently from Fathoming Rule 4.2, Fathoming Rule 4.3 explicitly considers the conflict constraints associated with the items in $X(L)$ and the set of remaining items $Y(L)$. Therefore, no dominance relation exists between the two rules. Clearly, at the root node of the BPC tree where no conflict constraints are present, Fathoming Rule 4.2 dominates Fathoming Rule 4.3.

To implement the pricing algorithm, we associate with an item $j \in \bar{N}$ a *bucket* $\mathcal{L}_j$ containing all labels $L = (j, X, \omega, \bar{c}, Y, \mathcal{R})$. The set of buckets is implemented by means of a doubly-linked list stored in one contiguous pre-allocated segment of memory where each node of the list, in addition to the link fields, contains the label data. Whenever the maximum allocation size is reached by the algorithm, the entire algorithm terminates prematurely. In order to reduce the space requirements, the sets X and Y and the resource vector $\mathcal{R}$ of a label are encoded by using a bit-mask representation where every single bit of a computer word is used separately to indicate whether an element is included in the set or not. Classical bit-set operations are used to implement the basic set operations.

The algorithm returns a set of patterns $\mathcal{P}' \subseteq \widehat{\mathcal{P}}$ containing the most negative reduced cost pattern and up to $\Delta - 1$ additional patterns with a negative reduced cost.

To speed up the computations, the lower-bounding functions $lb_1(\cdot)$ and $lb_2(\cdot)$ adopted in Fathoming Rules 4.2 and 4.3 are efficiently computed in the following way. For the computation of $lb_1(\cdot)$, we precompute the values of the recursive function $g(j, \omega) =$

$z(KP(\{j, \dots, \bar{n}\}, \omega, (\bar{w}_j, \dots, \bar{w}_{\bar{n}}), (-\pi_j, \dots, -\pi_{\bar{n}})))$ for all $j \in \bar{N}$, $\omega = 0, \dots, W$, using dynamic programming (Martello and Toth, 1990). This allows for computing the corresponding term in the bounding function $lb_1(L)$ as $g(j(L) + 1, W - \omega(L))$ with an overall number of operations equal to $O(|I|)$. For computing $lb_2(\cdot)$, we precompute the component-wise ratio between the profit vector $-\pi$ and the weight vector $\bar{w}$ and sort the corresponding values in non-increasing order. This allows for computing $lb_2(\cdot)$ within each label extension in $O(|\bar{N}|)$ operations.

To further reduce the number of generated labels, we also rely on the following dominance rule, also used by Wei et al. (2020):

Dominance Rule 4.4. Let L_1 and L_2 be two labels with $j(L_1) = j(L_2)$ and $\omega(L_1) \leq \omega(L_2)$. If

$$\bar{c}(L_1) - \sum_{\substack{\sigma \in \mathcal{R}: \\ \sigma(L_1)=1, \sigma(L_2)=0}} \rho(\sigma) \leq \bar{c}(L_2) - \sum_{j \in Y(L_2) \setminus Y(L_1)} \pi_j,$$

then label L_1 dominates label L_2 and, thus, label L_2 can be discarded.

The correctness of this rule follows from the observation that from the feasible extensions of L_2 we can construct feasible extensions for L_1 and that the reduced cost of L_1 cannot exceed that of L_2 .

4.2 Exact integer pattern enumeration algorithm

A classical result of integer programming (often called *reduced-cost fixing* and dating back to the seminal paper on the Traveling Salesman Problem by Dantzig et al. (1954)) is that, given an upper bound UB and the optimal solution value LB of the LP relaxation of a binary integer program, any variable with a reduced cost larger than $UB - LB$ cannot take a positive value in an optimal integer solution. Baldacci et al. (2008) were the first to show how this result can be leveraged in the context of a column generation method for the VRP to design a so-called *route enumeration* method. The same method was used by Contardo and Martinelli (2014) and Pessoa et al. (2020).

The idea is to run the CG algorithm until convergence and, then, apply a column enumeration algorithm (a variant of a list-based dynamic programming algorithm such as the one we presented previously) tasked with building all the columns with a reduced cost no larger than $UB - LB$. Such columns are used to populate a *Reduced Master Problem* (ReMP) which is then solved as an integer programming problem with a state-of-the-art MILP solver.

When translating this into the context of the BPP, we obtain a *pattern enumeration* method. Given a feasible (but not necessarily optimal) dual solution $(\pi^{\text{float}}, \rho^{\text{float}})$ of value LB of the dual of the LSCT and a valid upper bound UB , any optimal integer solution to the SC of cost less than UB cannot contain any pattern $S \in \mathcal{P}$ whose reduced cost computed with respect to the dual solution $(\pi^{\text{float}}, \rho^{\text{float}})$ is greater than or equal to $UB - LB$. Let $\widetilde{\mathcal{P}} \subset \mathcal{P}$ be the set of all the patterns that can be part of a solution that improves on UB (i.e., the set of all patterns of reduced cost strictly smaller than $UB - LB$). After building $\widetilde{\mathcal{P}}$ by enumeration, the BPP can be solved to optimality by formulating as ReMP a *reduced SC formulation* with explicit

integrality constraints featuring a variable per pattern in $\widetilde{\mathcal{P}}$ and then solving it with a MILP solver. This way, any optimal solution to the reduced SC formulation is, by construction, an optimal solution to the BPP.

In the BPP literature, a pattern enumeration method is used implicitly by Pessoa et al. (2021) when casting the BPP as an instance of the VRP and solving it with the VRPSolver algorithm that the authors propose. Crucially, though, their method (but also those we mentioned before for the VRP) relies on a floating-point MILP solver for solving the reduced SC formulation and, thus, the optimality of the solutions it finds *cannot be certified* due to the floating-point errors that can affect it.

To circumvent such a numerical issue and guarantee the construction of solutions to the reduced SC formulation that are numerically exact, in our algorithm we solve the reduced SC formulation with the very same numerically exact BPC method that we described before in which the pricing problem does not need to be solved.

The ReMP is built by applying the pattern enumeration method at the root node of the BPC tree after the computation of the root lower bound based on Algorithm 1 and then invoking the pricing algorithm with, as input, the final scaled integer solution $(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}})$ of value z^{int} to perform pattern enumeration, and in which Dominance Rule 4.4 is not used so as to be able to construct all the patterns with a reduced cost within the gap $UB - LB$.

As a further improvement, rather than just solving the reduced SC formulation we solve a *reduced-and-restricted* SC formulation in order to find solutions of value *strictly better* than UB . This is motivated by the fact that, in many instances of the BPP, it is often the case that the UB found at the root node already coincides with the optimal solution value of the problem (this is the case, e.g., of the ANI instances). The reduced-and-restricted SC formulation is obtained by imposing an objective cut thanks to which only solutions of value no larger than $UB - 1$ are sought. This way, while any optimal solution to such a reduced-and-restricted SC formulation is an optimal solution to the BPP, the infeasibility of the formulation proves that no such solution exists, thus allowing to conclude that UB is the optimal solution value of the problem.

When performing pattern enumeration in our BPC algorithm, we set the parameter Δ of Algorithm 1 to a user-defined parameter Δ_{PE} whose value is dictated by the amount of memory that is available (Δ_{PE} is set to 500,000 for the experiments reported in Section 6) and restrict the size of the reduced set of patterns $\widetilde{\mathcal{P}}$ to $|\widetilde{\mathcal{P}}| \leq \Delta_{\text{PE}}$. If $|\widetilde{\mathcal{P}}|$ is too large for the amount of memory that is available (i.e., if $|\widetilde{\mathcal{P}}| > \Delta_{\text{PE}}$), the reduced-and-restricted SC formulation is discarded and we resort to branching. On the contrary, if $|\widetilde{\mathcal{P}}| < \Delta_{\text{PE}}$, the reduced-and-restricted SC formulation is solved to optimality using a version of our BPC algorithm in which the pricing problem is disabled, thanks to which the BPC algorithm is turned into a branch-and-cut (BC) algorithm. This has several advantages, among which allowing for the separation of additional cuts such as the SR3 constraints without impacting the complexity of the pricing algorithm.

5 Root node optimality check, RF branching scheme, and overall exact method

In this section, we introduce a fathoming procedure to be applied at the root node of the enumeration tree and describe the branching procedures we adopt. The section is concluded with an overview of the exact method we use to solve the BPP.

Algorithm 2: BCCF: Numerically-Exact Algorithm for the BPP

Input: BPP instance, K , UB

Output: optimality, UB

```

begin
  // Initialization
1  optimality  $\leftarrow$  false; config  $\leftarrow$  BPC;
  // Root node solution
2  Execute Algorithm 1 to compute the scaled integer solution  $(\pi^{\text{int}}, \rho^{\text{int}})$  of cost  $z^{\text{dim}}$ ;
3  if  $\lceil z^{\text{dim}} \rceil \geq UB$  then optimality  $\leftarrow$  true; stop; ;
  // Pattern enumeration
4  Execute the pricing algorithm to perform pattern enumeration using the solution  $(\pi^{\text{int}}, \rho^{\text{int}})$  and
   let  $\widetilde{\mathcal{P}}$  be the resulting set of patterns;
  // Turn BPC into BC if pattern enumeration is successful
5  if  $|\widetilde{\mathcal{P}}| < \Delta_{PE}$  then config  $\leftarrow$  BC; ;
  // Root node optimality check
6  Execute RNOC based on the value of config (BPC or BC);
7  if RNOC is successful then optimality  $\leftarrow$  true; stop; ;
  // BPC or BC approach
8  Perform branch and cut or branch and price and cut based on the value of config (BPC or BC)
   using Algorithm 1 at each node of the enumeration tree and adopting RF branching;
9 end

```

Our preliminary experiments revealed that, after solving the root node either in the BPC or in the BC algorithms, the adoption of a RF branching scheme is not always very effective because (as it is known for BPP instances) it may lead to very large enumeration trees. For this reason, before resorting to RF branching, we apply an additional *root node optimality check* (RNOC) procedure. The procedure aims at computing at the root node a dual bound strong enough to prove (if this is the case) that the incumbent of value UB is optimal, as this would allow the algorithm to terminate.

RNOC takes as input a fractional solution ξ to the RMP and for each pair of items $i, j \in \overline{N}$ with $i < j$ it computes $\nu_{ij} = \sum_{\substack{S \in \widehat{\mathcal{P}} \\ i, j \in S}} \xi_S$, which coincides with the sum of the values of the ξ_S variables whose pattern $S \in \widehat{\mathcal{P}}$ contains both i and j . Starting from $\nu_{ij} = 0$ for all $i, j \in \overline{N}$, the value is computed efficiently by iterating over the patterns $S \in \widehat{\mathcal{P}}$ and, for each pair of items $i, j \in S$, letting $\nu_{ij} = \nu_{ij} + \xi_S$. These values are used not only within RNOC, but also for RF branching if the former fails to prove the optimality of the instance.

The RNOC procedure operates differently based on whether the algorithm is running in the BPC or BC case. In the BPC case, the RNOC is used to find (if any) an RF branching pair $i, j \in \overline{N}$ with ν_{ij} taking a fractional value such that the dual bound that is calculated after solving the

two subproblems that are obtained by branching over such a fractional pair (which is equal to the smallest of the bounds calculated with the column-and-row-generation procedure in each of the two child nodes) is equal to UB . Finding such a pair provides a certificate of optimality of the current solution, allowing the BPC algorithm to terminate at the root node.

In the BC case, we use the RNOC to find (if any) an RF pair $i, j \in \overline{N}$ with ν_{ij} taking a fractional value leading to two subproblems both of which are infeasible. Thanks to the presence of the objective cut in the formulation, finding such a pair of items guarantees that no solution of value strictly better than UB exists, thus allowing the BC algorithm to terminate.

The RNOC procedure iterates over all the pairs with a fractional value of ν_{ij} until either it manages to prove the optimality of the current solution or every such pair of items has been considered. In the latter case, RNOC fails and RF branching is applied in both the BPC and the BC algorithms by selecting a pair of items that are fractionally covered and with ν_{ij} closer to 0.5. After branching, the columns that are infeasible for the branching constraints are removed from the RMP.

As shown in more details in Section 6, our experiments revealed that the RNOC procedure allows to solve a good fraction of instances to proven optimality.

The main steps of our exact algorithm, to which we refer as **BCCF** (named after the authors of this paper), are summarized in Algorithm 2. After computing the root node solution, the algorithm performs pattern enumeration and, based on whether the latter was successful or not, selects whether the pricing component should be switched off (BC option) or not (BPC option) by setting the value of the variable `config`. Then, the algorithm executes the RNOC procedure and, if the latter is not successful, the given BPP instance is solved to optimality using the algorithm that was selected with `config` (either BPC or BC). In our numerical experiments, we impose a time limit for solving the root node and for running the BPC and BC algorithms.

6 Computational experiments

We assess in this section the computational performance of our numerically-exact algorithm **BCCF** based on BPC and pattern-enumeration. The main goal of this computational study is assessing the role that each of the key components of **BCCF** plays towards solving BPP instances to optimality in a numerically exact way. For comparison purposes, we contrast the computational behavior of **BCCF** with three state-of-art BPP algorithms from the literature—**EXM**, **VRP solver**, and **NF-F**. We remark that, differently from **BCCF**, none of these algorithms is numerically exact due to either employing non-safe bounds (**EXM**) or resorting to a MILP solver to carry out certain operations (**VRP solver** and **NF-F**). The files needed to run **BCCF** can be downloaded from the website: <https://github.com/stefanoconiglio/A-Numerically-Exact-Algorithm-for-the-Bin-Packing-Problem>.

6.1 BPP benchmark sets

Among the many instances proposed for the BPP (an extensive description of which is provided by [Delorme et al. \(2016\)](#)), in this computational study we focus on non-IRUP instances. As mentioned in Section 1, $z(\text{BPP}) = \lceil z(\text{LSC}) \rceil + 1$ holds on every non-IRUP instance known in the literature. Therefore, computing $z(\text{LSC})$ with a column generation method does not suffice to calculate $z(\text{BPP})$ and more sophisticated techniques (the generation of SR3 constraints, branching, or pattern enumeration) are needed. In particular, if $\lceil z(\text{LSC}) \rceil = z(\text{LSC})$ holds, then any strictly positive bound improvement over $z(\text{LSC})$ suffices to calculate $z(\text{BPP})$.

In our study, we focus on the instances of the Augmented Non-IRUP (**ANI**) class ([Delorme et al., 2016](#)), which comprises 250 instances with a number of items n ranging from 201 to 1,002 and a capacity c ranging from 2,500 to 80,000. These instances enjoy two properties: they satisfy $z(\text{BPP}) = \lceil z(\text{LSC}) \rceil + 1$ with $z(\text{LSC}) = \lceil z(\text{LSC}) \rceil$ and, for each of them, a solution whose value coincides with the optimal solution value can be found by running the best-fit algorithm. The challenge for this class is therefore proving the optimality of the greedy solution.

Crucially (as indicated before), the **ANI** instances are particularly challenging due to numerical difficulties that arise when solving the LSC and LSCT (see, e.g., [Pessoa et al., 2021](#)). In particular, while $z(\text{LSC}) = \lceil z(\text{LSC}) \rceil = z(\text{BPP}) - 1$ holds in infinite-precision arithmetic, it is very often the case that when numerically-safe calculations are carried out in floating-point arithmetic $z(\text{LSC}) < \lceil z(\text{LSC}) \rceil$ holds, implying that a strictly positive bound improvement over $z(\text{LSC})$ may not suffice to calculate $z(\text{BPP})$.

6.2 Setup

We rely on two LP solvers in our implementation: **Gurobi** 9.5.1 ([Gurobi Optimization, LLC, 2022](#)) and **SoPlex** 5.0.1 ([Wunderling, 1996](#); [Gleixner et al., 2012, 2016](#)). **Gurobi**'s LP solver is used in phase I of Algorithm 1 and in the enumeration trees associated with the BPC and the BC approaches. **SoPlex** is used in phase II of Algorithm 1 and it is set to work in infinite-precision. At the root node, we found to be computationally advantageous to set the initial value of ϵ_0 to 10^{-6} , use **Gurobi** as long as $\epsilon > 0$, and switch to **SoPlex** when $\epsilon = 0$. In the enumeration trees associated with the BPC and the BC approaches, phase II of Algorithm 1 is not used, and we set the value of ϵ_0 to 10^{-9} , so that **Gurobi** works within its smallest optimality tolerance. The parameter Δ of the pricing algorithm used to generate negative reduced cost patterns is set to 10 (thus, no more than 10 patterns are generated per iteration). As a result of a set of preliminary experiments, we set the value of K to $2^{44} \simeq 10^{13}$. Section 6.6 describes how varying K impacts the algorithm's effectiveness.

At the root node of the BPC tree, we rely on the greedy *best-fit* algorithm to build an approximate solution in a very short amount of time ([Dósa and Sgall, 2014](#)).

By construction, on the **ANI** instances the LP bound of the classical formulation of the BPP featuring $O(n(n+1))$ binary variables (which can be computed in closed form in $O(n)$ as $\frac{1}{c} \sum_{j \in N} w_j$) coincides with $z(\text{LSC})$ ([Delorme et al., 2016](#)). Improving on this bound by either

branching or by cut generation is therefore key to solving these instances. In order to obtain a bound as strong as possible, in our experiments we separate the SR3 constraints not only at the root node but also at every node of the BPC tree. As the computing time needed to separate the SR3 constraints increases w.r.t. the number of items and the number of generated patterns, in our experiments we limit their number to no more than 50.

Since, as explained before, the greedy heuristic is always able to find an optimal solution for the ANI instances, we do not rely on any additional heuristic methods for solving them. We remark, however, that BCCF is equipped with different heuristics, including: 1) a *diving heuristic* based on a depth-first heuristic search in the branch-and-price tree that relies on a limited backtracking as a diversification strategy and on a limited discrepancy search method similar to those of Sadykov and Vanderbeck (2013) and Wei et al. (2020); 2) the greedy *best-fit* algorithm; 3) an LP-based heuristic by which the patterns with ξ_S larger than a given threshold (up to a certain maximum cardinality) are added to the solution and the yet uncovered items are packed greedily with the best-fit algorithm.

The experiments with BCCF are run on the IRIDIS 5.0 High Performance Computing Facility of the University of Southampton, relying on a cluster of compute nodes equipped with dual Intel(R) Xeon(R) Gold 6130 CPUs @ 2.10GHz and 192 GB of DDR2 RAM using a single thread per experiment.¹ Our source code is written in C and C++, and compiled with gcc 11.1.0 with the -O3 optimization flag. In line with the other computational studies in the literature, all the experiments are run within a time limit of 3,600 seconds per instance.

6.3 Comparison with state-of-the-art not numerically-exact algorithms

For comparison purposes, we begin our analysis by contrasting BCCF with the following three state-of-the-art BPP algorithms (none of which is numerically-exact):

- **EXM**: BPC algorithm proposed by Wei et al. (2020), run sequentially on an Intel Xeon E5-1603 at 3.10 GHz equipped with 8 GB of RAM. The algorithm is not numerically exact due to relying on a non-safe Lagrangian bound and a non-safe floating-point pricing algorithm.
- **VRP solver**: BPC algorithm proposed by Pessoa et al. (2021), run in parallel with 8 computations at a time on a 2 Deca-core Ivy-Bridge Haswell Intel Xeon E5-2680 v4 server running at 2.50 GHz equipped with 128 GB of RAM. The algorithm computes safe dual bounds during the enumeration tree based on a safe bound obtained with a procedure similar to the one of Held et al. (2012) (whose details, though, are not given in the paper of Pessoa et al. (2021)), but it also adopts a non-safe MILP solver to solve the restricted master problem. As in the paper no detailed results are reported to indicate for which instances the MILP solver was used, it is not possible to verify how many of the solutions found by VRP solver are numerically exact.

¹https://en.wikichip.org/wiki/intel/xeon_gold/6130

Table 1: Number of instances solved exactly to optimality by BCCF and average computing time for the ANI class, compared with the results obtained with EXM, VRPsolver, and NF-F.

Group	inst.	BCCF		EXM		VRPsolver		NF-F	
		opt	time [s]	solved	time [s]	solved	time [s]	solved	time [s]
ANI-201	50	50	13.6	50	13.9	50	16.7	50	3.0
ANI-402	50	50	308.2	47	436.2	50	96.0	50	24.9
ANI-600	50	25	1,931.5	0	tl	3	3,512.5	50	140.7
ANI-801	50	3	3,352.7	0	tl	0	tl	49	393.2
ANI-1002	50	0	tl	0	tl	0	tl	43	1,302.5
tot	250	128		97		103		242	

- NF-F: algorithm based on the *network-flow framework* proposed by [de Lima et al. \(2021\)](#) to solve the integer counterpart to a Dantzig-Wolfe decomposition via a path-flow and arc-flow model, which the authors also use to solve BPP instances; it is run on a computer equipped with an Intel Xeon E3-1245 v5 at 3.50 GHz and 32 GB of RAM, with a single-thread limit. The algorithm computes safe dual bounds based on the scaling technique proposed by [Held et al. \(2012\)](#), but it also uses a non-safe MILP solver as a key ingredient in the branching scheme to fathom the left child at each level of the enumeration tree and also at each leaf node.

The results obtained on the ANI instances are reported in Table 1, aggregated by the number of items n into five groups: ANI-201 (201 items), ANI-402 (402 items), ANI-600 (600 items), ANI-801 (801 items), and ANI-1002 (1002 items). Each group comprises 50 instances.

The table reports the number of instances solved by BCCF (column “opt”) by constructing an exact proof of optimality and the average computing time in seconds (column “time [s]”). For the other algorithms, it reports the number of instances for which each algorithm terminated with a solution that it claimed to be optimal (column “solved”) and the corresponding average computing time (column “time[s]”). In line with other works in the BPP literature, the average time is computed across all the instances, including those for which the time limit of 3,600 seconds is reached. In the table, “tl” indicates a method that reached the time limit for all of the instances in a group.

As the computational environments in which the different algorithms were run is heterogeneous, the solution times of the algorithms cannot be directly compared with absolute precision. Therefore, our comparison with respect to the literature will be based on the number of instances solved exactly to optimality by BCCF and on the number of instances for which the other algorithms terminated with a solution they deemed to be optimal.

It is worth noting that, in our preliminary experiments with Algorithm 1 and in line with what is reported by [Pessoa et al. \(2020\)](#), we observed that, on the instances with 600 items or more and with Gurobi working within its smallest optimality tolerance (10^{-9}), the pricing problem finds columns with a negative reduced cost that the LP solver does not include in the basis due to the absolute value of the reduced cost being smaller than the optimality tolerance the solver adopts

Table 2: Impact of the different components of BCCF (first part)

Group	inst.	Phase I		SR3		Phase II			Pattern Enumeration		
		col	time [s]	opt	time [s]	#	col	time [s]	#	col	time [s]
ANI-201	50	279.7	1.6	41	1.1	9	121.7	0.8	1	25,235.0	31.5
ANI-402	50	489.5	21.5	18	20.8	32	324.1	4.8	22	2,219.5	0.5
ANI-600	50	660.1	111.6	0	117.0	47	475.6	13.9	25	2,145.0	0.9
ANI-801	50	827.4	360.2	0	340.1	19	537.0	28.0	3	1,809.3	1.3
ANI-1002	50	1,300.5	848.6	0	775.0	18	786.7	80.1	-	-	-
Tot	250			59		125			51		

(**OptimalityTol**). The final solution obtained by the column-and-row-generation algorithm is therefore not dual feasible. Thus, the numerically-safe bound obtained by the scaling procedure can be rather weak. Therefore, we believe that the results of algorithms such as **VRP solver** and **NF-F** that employ MILP solvers relying on a finite-precision LP algorithm could be heavily affected by numerical errors when attempting to solve these instances.

As Table 1 shows, **BCCF** solves 128 instances out of the 250 instances of the ANI class. The algorithm outperforms both **EXM** and **VRP solver** in terms of the number of instances solved to optimality and, on the instances with up to 402 items, it also solves the same instances solved by **NF-F**. For the larger instances with 600 and 801 items, **BCCF** largely outperforms **EXM** and **VRP solver** by solving 25 instances with 601 items and 3 with 802 items, whereas these two methods combined can only solve 3 with 601 items and none with 802 items. No instance with 1002 items is solved by either **BCCF**, **EXM**, or **VRP solver**. **NF-F** solves (albeit without a guarantee of numerical exactness) almost all the instances with more than 600 items. However, the solutions it produces are not certified to be optimal within an infinite precision, a property that is enjoyed by the solutions produced by **BCCF**.

6.4 Analysis of the role played by the different components of our algorithm

Tables 2, 3, and 4 illustrate the impact of the different components of **BCCF**. In the tables, the instances are grouped as in Table 1.

Under the heading “Phase I”, Table 2 reports the average number of columns (“col”) and the average computing time in seconds (“time [s]”) of phase I at the root node of the enumeration tree, also factoring in the time taken by **Gurobi** to solve the LP relaxations of the RMP, the time taken by the DP algorithm to generate new columns, and the time taken to separate SR3 inequalities. Under the heading “SR3”, the table indicates how many instances are solved to optimality thanks to the generation of SR3 cuts (“opt”) and the average time spent by separating them (“time [s]”) throughout the entire enumeration tree. The generation of SR3 constraints is not carried out in phase II (performed only at the root node) as extensive preliminary computational results indicated that it was not effective.

Under the heading “Phase II”, Table 2 reports the average number of columns (“col”), the average computing time in seconds (“time [s]”), which factors in the time spent by **SoPlex** to solve the

Table 3: Impact of the different components of BCCF (second part)

Group	inst.	Phase I			Phase II		
		iter	time LP [s]	time DP [s]	iter	time LP [s]	time DP [s]
ANI-201	50	56.20	0.23	0.49	4.42	0.13	0.01
ANI-402	50	88.52	1.24	1.90	31.18	2.65	0.33
ANI-600	50	127.96	4.23	5.71	63.80	17.94	1.69
ANI-801	50	176.50	10.46	24.77	74.10	86.54	4.47
ANI-1002	50	257.72	28.87	78.74	104.96	128.34	14.01

LP relaxations of the RMP and the time taken by the DP algorithm to generate new columns, and the number of times (“#”) Algorithm 1 enters phase II.

The “Pattern Enumeration” section of Table 2 reports the number of instances for which pattern enumeration was successfully performed (i.e., the number of times the full set of columns within the gap was generated without exceeding any memory limitations) (“#”), the number of patterns it generated (“col”), and the average computing time spent by the label-setting algorithm. The pattern-enumeration method runs out of memory for 8, 10, 25, 46, and 49 instances with 201, 402, 600, 801 and 1002 items, respectively.

Overall, Table 2 shows that, while relying on a much less efficient solver than phase I does, phase II is fairly fast due to generating a much smaller (by about an order of magnitude) number of columns. It also shows that the SR3 constraints are useful mostly on the smaller instances, whereas it clearly indicates that the pattern enumeration procedure is responsible for solving a large number of instances.

In Table 3, we report additional computational details of the two phases of BCCF collected during their execution at the root node and focusing on the times spent to solve the LP relaxations of the RMP and to execute the DP algorithm. For both phases, we report the average number of iterations (“iter”), the average time spent to solve the LP relaxations of the RMP by **Gurobi** and **SoPlex**, and the average time spent by the DP algorithm to generate new columns.

Regarding the number of iterations, Table 3 shows that, starting from instances with 400 items, phase II requires on average less than half the number of iterations that phase I does. The average computing time taken by the DP algorithm per execution or iteration is of 0.1 second for phase I and of 0.04 second for phase II. As expected, the average time per iteration taken by **SoPlex** is larger than the time taken by **Gurobi**. This result can be due to two factors: (i) the larger size of the RMP when phase II starts and, as expected, (ii) the fact that **SoPlex** is used as an exact LP solver. Across all the instances, **Gurobi** takes on average 0.04 second per iteration while **SoPlex** takes 0.55 second per iteration, thus resulting almost an order of magnitude slower.

Table 4 provides details about the effectiveness of adopting the RNOC fathoming procedure within BCCF as well as on the instances that are solved by solving the reduced SC formulation with regular RF branching operations and not at the root node thanks to RNOC. More specifically,

Table 4: Impact of the different components of BCCF (third part)

Group	inst.	RNOC				RF branching		
		opt	time [s]	pairs	branches	opt	nodes	time [s]
ANI-201	50	8	16.8	405.4	44.3	1	40.0	54.7
ANI-402	50	25	232.2	207.2	53.6	7	5,788.0	1,484.4
ANI-600	50	25	1,386.9	83.8	12.7	0		
ANI-801	50	3	2,852.7	76.3	11.3	0		
ANI-1002	50	-	2,556.1			0		
Tot	250	61				8		

under the heading “RNOC”, the table reports the number of instances solved to optimality (“opt”) thanks to RNOC (for both the BPC and BC algorithms), the average number of (fractional) pairs of item contained in the instance at the root node (“pair”), and the average number of fractional pairs that RNOC considers before either managing to prove the optimality of the incumbent or failing to do so (thereby considering all the pairs), after which RF branching is started. The RNOC procedure is applied either at the root node of the BC algorithm used for solving the reduced SC formulation obtained after applying the pattern enumeration method or at the root node of the BPC algorithm which is run whenever the pattern-enumeration procedure fails to build the reduced SC formulation due to memory limitations. Under the heading “RF branching”, the table reports the number of instances solved to optimality by either the BC or the BPC algorithm with RF branching (“opt”) beyond the root node (i.e., when the RNOC procedure is unsuccessful), the number of nodes (“nodes”), and the computing time (“time [s]”).

Our results indicate that the average number of fractional pairs present in the optimal LP solution of the root node, while substantial, decreases w.r.t. the number of items and that the actual number of pairs explored by RNOC when successful is much smaller, ranging from 1/4th to 1/10th of the total number of pairs. Table 4 clearly shows that, while the time RNOC takes can be substantial (taking from 50% to 75% of the overall computing time on the ANI-600, ANI-801, and ANI-1002 instances), RNOC is very effective as it is responsible for solving 61 instances out of the 128 that BCCF solves to optimality. It also shows that, if RNOC fails, RF branching is still useful, as it is responsible for solving 8 instances.

Relying on Tables 2, 3, and 4, we now analyze the results obtained by BCCF on each individual group of instances.

- **ANI-201:** BCCF solves 41 of the 50 instances with 201 items at the root node thanks to the separation of the SR3 inequalities. For all such instances, $z(\text{LSCT})$ takes values between 65.0000007774641 and 65.0000276484105 (with a value of $K \simeq 10^{13}$). Since, with infinite precision, we have $z(\text{LSC}) = 65$ and $z(\text{BPP}) = 66$, such an improvement suffices to prove the optimality of the greedy solution on each of these 41 instances. A total of 8 instances are solved at the root node of either the BC or the BPC tree by RNOC, whereas one instance is solved in 54.7 seconds thanks to RF branching.
- **ANI-402:** BCCF manages to solve 18 instances with 402 items at the root node thanks to

Table 5: Summary of the performance of different versions of BCCF.

Group	inst.	BCCF		No SR3		No RNOC		No Patt. Enum.	
		opt	time [s]	opt	time [s]	opt	time [s]	opt	time [s]
ANI-201	50	50	13.6	50	112.3	50	16.5	50	7.7
ANI-402	50	50	308.2	45	494.2	47	427.0	44	695.3
ANI-600	50	25	1931.5	25	1851.5	19	2347.7	0	tl
ANI-801	50	3	3352.7	3	3392.4	3	3358.7	0	tl
ANI-1002	50	0	tl	0	tl	0	tl	0	tl
Tot		128		123		119		94	

the separation of SR3 inequalities. The pattern enumeration procedure runs successfully on 22 instances. A total of 25 instances are solved by RNOC, and 7 instances are solved by RF branching outside of the root node.

- ANI-600: BCCF solves 25 of the 50 instances with 600 items, all of which thanks to the pattern enumeration procedure and the RNOC procedure applied at the root node of the BC algorithm.
- ANI-801: BCCF solves 3 instances of the 50 instances with 801 items, namely, 801_40000_NR_0, 801_40000_NR_11, and 801_40000_NR_27, all of which thanks to the pattern enumeration and the RNOC procedure. On them, the pattern-enumeration procedure takes less than two seconds, generating approximatively 1800 columns per instance. For all these instances, the RNOC procedure proves the infeasibility of the reduced SC formulation in 2852 seconds on average without branching.

6.5 Ablative study of the effectiveness of the main components of our algorithm

The effectiveness of the main components of BCCF are summarized in Table 5. The results are obtained with three variants of BCCF where one of the following components is disabled: separation of SR3 cuts (“No SR3”), RNOC fathoming procedure (“No RNOC”), or pattern enumeration (“No Patt. Enum.”).

The table shows that each component helps to improve the performance of BCCF. In particular, it shows that the pattern enumeration method combined with the computation of extended-precision safe dual bounds plays the most important role as it allows for solving 34 more instances than the version of BCCF in which such a feature is disabled. In particular, this feature is responsible for solving all 25 instances with 600 items that the full version of BCCF manages to solve. It also shows that SR3 constraints and RNOC play an important, albeit smaller, role, as 5 fewer instances are solved when the SR3 constraints are not separated and 9 fewer instances are solved when RNOC is disabled.

6.6 Analysis of the impact of the scaling factor K

In this section, we describe how the scaling factor K can be determined. We also analyze the impact of varying K on the numerical precision of the dual bounds calculated within BCCF (see Proposition 3.2) and its impact on the effectiveness of the main components of the algorithm.

Let $I_{\min} < 0$ and $I_{\max} > 0$ be the minimum and maximum integer values, respectively, that can be represented by the data type used for storing integer values. Standard implementations (including ours), rely on 64-bit signed integers, for which $I_{\min} = -9,223,372,036,854,775,808$ and $I_{\max} = 9,223,372,036,854,775,807$ (inclusive). For an error-free computation of the scaled integer reduced cost $\bar{c}_S(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}}, K)$, we need to ensure

$$I_{\min} \leq \sum_{j \in S} \pi_j^{\text{int}} + \sum_{T \in \overline{\mathcal{T}}(S)} \rho_T^{\text{int}} \leq I_{\max}.$$

For this to hold, we must guarantee that the largest scaled integer reduced cost value that can be computed be less than or equal to $I_{\max}$ and that the smallest value be greater than or equal to $I_{\min}$. Since $\boldsymbol{\pi}^{\text{int}} \geq \mathbf{0}$ and $\boldsymbol{\rho}^{\text{int}} \leq \mathbf{0}$, we can restrict ourselves to the following sufficient condition:

$$\sum_{j \in S} \pi_j^{\text{int}} \leq I_{\max} \text{ and } \sum_{T \in \overline{\mathcal{T}}(S)} \rho_T^{\text{int}} \geq I_{\min}.$$

Let $\hat{p}$ and $\hat{q}$ be two upper bounds on, respectively, the number of superitems and the number of SR3 inequalities associated with the RMP (i.e., $\hat{p} \geq \bar{n}$ and $\hat{q} \geq |\overline{\mathcal{T}}|$) and assume $(\boldsymbol{\pi}^{\text{float}}, |\boldsymbol{\rho}^{\text{float}}|) \in [0, 1]^{\bar{n}} \times [0, 1]^{|\overline{\mathcal{T}}|}$ (we observed that this is always the case in our numerical experiments).

By construction, this implies that no entry of $(\boldsymbol{\pi}^{\text{int}}, \boldsymbol{\rho}^{\text{int}})$ is larger (in absolute value) than K . To avoid integer overflows during the solution of the pricing problem, K must therefore be chosen in such a way that the following holds:

$$K \leq \min \left\{ \frac{I_{\max}}{\hat{p}}, -\frac{I_{\min}}{\hat{q}} \right\}.$$

The occurrence of integer overflows is explicitly checked in our code. In case of their occurrence, the algorithm is terminated without returning a solution. In our experiments, such a case was never witnessed.

The impact of varying the scaling factor K on the performance of BCCF is reported in Table 6. The table reports the number of instances solved to optimality (“opt”) as a function of the value of the precision parameter K . It also focuses on the pattern enumeration method, indicating the number of instances for which the method was run successfully without exceeding the memory limitations (“#”), and the average number of patterns (“cols”), and labels (“labels”) that it generates for different values of the scaling factor K .

Table 6 clearly shows the positive impact of increasing K . With the largest K , BCCF solves 36 more instances than with the smallest K . With a higher K , improved dual bounds can be computed and the pattern enumeration procedure, whose effectiveness relies on the value of the

Table 6: Impact of varying the scaling factor K

K	opt	Pattern Enumeration		
		#	cols	labels
$2^{44} \simeq 10^{13}$	128	51	2,610.1	321,763.8
$2^{37} \simeq 10^{11}$	123	50	32,495.8	1,023,669.1
$2^{30} \simeq 10^9$	92	16	116,741.4	2,656,733.9

restricted gap, becomes more effective. This is testified by the average number of patterns that are generated when varying the value of K .

It must be noted, though, that the performance in terms of number of labels generated by the DP algorithm used to enumerate the patterns depends not only on the restricted gap but also on the values of the dual variables associated with the dual bound. In practice, we observe that the distribution of the dual variables strongly affects the number of labels generated by the DP algorithm and that enumerating patterns based on a smaller restricted gap does not necessarily result in a smaller number of DP labels. Moreover, we observe that **SoPlex** may suffer from numerical instabilities with higher values of K , leading to an abnormal termination of phase II (this happens when, after building an “optimal” floating-point solution, **SoPlex** tries to build an optimal rational one by a rational implementation of the simplex method in which the floating-point solution is used to provide a starting basic solution). For these reasons, in our experiments a few instances that are solved with a lower value of K cannot be solved with a larger value.

Overall, Table 6 illustrates how crucial the calculation of (safe) dual bounds with a precision (10^{13}) higher than what commercial solvers allow for (10^9) is, validating the relevance of the two-phase column-and-row generation method we proposed.

6.7 Optimal rational solutions to the FBPP

As a byproduct of our work, we computed the first (to our knowledge) optimal rational dual solutions to the LP relaxation of the SC formulation of the BPP (the FBPP) for 189 instances of the ANI class.

The solutions are certified to be exact by a rational implementation of the pricing algorithm (in its lookup-table version, which suffices for solving the pricing problem when no SR3 constraints have been generated and no branching operations have taken place). Such a version, while computationally inefficient due to operating in rational arithmetic, allows for manipulating the dual variables with no restrictions on their precision (i.e., with $K = \infty$). As such, whenever an optimal dual solution is given as input to the algorithm, the latter can certify its optimality with infinite precision.

The solutions can be found at the address <https://github.com/stefanoconiglio/A-Numerically-Exact-Algorithm-for-the-Bin-Packing-Problem>. The repository contains solutions to 50/50 instances with 201 items, 50/50 instances with 402 items, 50/50 instances

with 600 items, 24/50 instances with 801 items, and 15/50 instances with 1002 items.

7 Conclusions

We proposed the first numerically-exact algorithm for the bin packing problem (BPP). Key to the method is the calculation of extended-precision numerically-safe dual bounds with a two-phase column-and-row generation method based on a dual-variable scaling technique that combines a floating-point (finite precision) linear programming solver with a rational (infinite precision) one. To the best of our knowledge, rational solvers had never been successfully used before in the context of a column generation method. The method relies on a suitably-designed exact integer (fixed-point) pricing algorithm which is designed to also perform pattern enumeration, i.e., to build a reduced master (set covering) problem containing a superset of the columns (patterns) that are featured in an optimal integer solution to the BPP.

Our numerically-exact algorithm was tested on the class of Augmented Non-IRUP (ANI) instances (which are affected by notorious numerical difficulties) and its performance compared with state-of-the-art not numerically-exact algorithms for the BPP. The results we obtained show that the our exact algorithm solves more instances than the state-of-the-art not numerically-exact algorithms based on branch-and-cut-and-price that rely on a set-covering formulation of the BPP. Thanks to the extended precision in which our algorithm works, all the solutions it produces on the instances we tested are certified to be optimal within an infinite precision, a property that is not enjoyed by the solutions produced by the other algorithms.

The framework we introduced with our algorithm can be of interest for the design of numerically-exact algorithms of a similar nature to solve a large array of combinatorial optimization problems that are typically tackled via branch-price-and-cut techniques that feature a set covering/partitioning problem as the master problem.

Among many interesting possibilities, future works include the design of a family of modified master problems designed to generate a set of dual variables with a distribution that is conducive to generating a small number of patterns within the pattern-enumeration method, possibly at the cost of worsening the dual bound by a small additive term.

Acknowledgments

The authors would like to thank the anonymous reviewers for their helpful suggestions and very thorough review of the paper. The authors are also thankful to the University of Southampton and, in particular, to the School of Mathematical Sciences and the Operational Research Group for kindly allowing them to use the university's high-performance computing facilities.

References

Applegate, D. L., Cook, W., Dash, S. and Espinoza, D. G. (2007), Exact solutions to linear programming problems, *Operations Research Letters* **35**(6), 693–699.

- Baldacci, R., Christofides, N. and Mingozzi, A. (2008), An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts, *Mathematical Programming* **115**, 351–385.
- Belov, G. and Scheithauer, G. (2006), A branch-and-cut-and-price algorithm for one-dimensional stock cutting and two-dimensional two-stage cutting, *European Journal of Operational Research* **171**(1), 85–106.
- Caprara, A., Dell’Amico, M., Díaz-Díaz, J. C., Iori, M. and Rizzi, R. (2015), Friendly bin packing instances without integer round-up property, *Mathematical Programming* **150**(1), 5–17.
- Coniglio, S., D’Andreagiovanni, F. and Furini, F. (2019), A lexicographic pricer for the fractional bin packing problem, *Operations Research Letters* **47**(6), 622–628.
- Contardo, C. and Martinelli, R. (2014), A new exact algorithm for the multi-depot vehicle routing problem under capacity and route length constraints, *Discrete Optimization* **12**, 129–146.
- Cook, W., Dash, S., Fukasawa, R. and Goycoolea, M. (2009), Numerically safe Gomory mixed-integer cuts, *INFORMS Journal on Computing* **21**(4), 641–649.
- Cook, W., Koch, T., Steffy, D. E. and Wolter, K. (2013), A hybrid branch-and-bound approach for exact rational mixed-integer programming, *Mathematical Programming Computation* **5**(3), 305–344.
- Cook, W., Koch, T., Steffy, D. and Wolter, K. (2011), *An Exact Rational Mixed-Integer Programming Solver*, Vol. 6655 of *Lecture Notes in Computer Science*.
- Cornuéjols, G., Margot, F. and Nannicini, G. (2013), On the safety of Gomory cut generators, *Mathematical Programming Computation* **5**(4), 345–395.
- Dantzig, G., Fulkerson, R. and Johnson, S. (1954), Solution of a large-scale traveling-salesman problem, *Journal of the Operations Research Society of America* **2**(4), 393–410.
- de Lima, V. L., Iori, M. and Miyazawa, F. K. (2021), New exact techniques applied to a class of network flow formulations, in M. Singh and D. P. Williamson, eds, ‘Integer Programming and Combinatorial Optimization’, Springer International Publishing, Cham, pp. 178–192.
- de Lima, V. L., Iori, M. and Miyazawa, F. K. (2022), Exact solution of network flow models with strong relaxations, *Mathematical Programming* pp. 1–34.
- Delorme, M., Iori, M. and Martello, S. (2016), Bin packing and cutting stock problems: Mathematical models and exact algorithms, *European Journal of Operational Research* **255**(1), 1 – 20.
- Dósa, G. and Sgall, J. (2014), Optimal analysis of best fit bin packing, in ‘International Colloquium on Automata, Languages, and Programming’, Springer, pp. 429–441.
- Eifler, L. and Gleixner, A. (2022), A computational status update for exact rational mixed integer programming, *Mathematical Programming* .
URL: <https://doi.org/10.1007/s10107-021-01749-5>
- Eisenbrand, F., Pálvölgyi, D. and Rothvoß, T. (2013), Bin packing via discrepancy of permutations, *ACM Transactions on Algorithms (TALG)* **9**(3), 1–15.
- Farley, A. A. (1990), A note on bounding a class of linear programming problems, including cutting stock problems, *Operations Research* **38**(5), 922–923.
- Fukasawa, R. and Goycoolea, M. (2011), On the exact separation of mixed integer knapsack cuts, *Mathematical Programming* **128**(1), 19–41.
- Fukasawa, R. and Poirrier, L. (2017), Numerically safe lower bounds for the capacitated vehicle routing problem, *INFORMS Journal on Computing* **29**(3), 544–557.
- Gilmore, P. C. and Gomory, R. E. (1961), A linear programming approach to the cutting-stock problem, *Operations Research* **9**(6), 849–859.

- Gleixner, A. M., Steffy, D. E. and Wolter, K. (2012), Improving the Accuracy of Linear Programming Solvers with Iterative Refinement, ZIB-Report 12-19, Zuse Institute Berlin. Accepted for publication in proceedings of ISSAC 2012: 37th International Symposium on Symbolic and Algebraic Computation.
- Gleixner, A. M., Steffy, D. E. and Wolter, K. (2016), Iterative refinement for linear programming, *INFORMS Journal on Computing* **28**(3), 449–464.
- Gurobi Optimization, LLC (2022), ‘Gurobi Optimizer Reference Manual’.
URL: <https://www.gurobi.com>
- Held, S., Cook, W. J. and Sewell, E. C. (2012), Maximum-weight stable sets and safe lower bounds for graph coloring, *Mathematical Programming Computation* **4**(4), 363–381.
- Jepsen, M., Petersen, B., Spoorendonk, S. and Pisinger, D. (2008), Subset-row inequalities applied to the vehicle-routing problem with time windows, *Operations Research* **56**(2), 497–511.
- Karmarkar, N. and Karp, R. M. (1982), An efficient approximation scheme for the one-dimensional bin-packing problem, in ‘23rd Annual Symposium on Foundations of Computer Science (SFCS 1982)’, p. 312–320.
- Kartak, V. M., Ripatti, A. V., Scheithauer, G. and Kurz, S. (2015), Minimal proper non-IRUP instances of the one-dimensional cutting stock problem, *Discrete Applied Mathematics* **187**, 120–129.
- Lübbecke, M. and Desrosiers, J. (2005), Selected topics in column generation, *Operations Research* **53**(6), 1007–1023.
- Martello, S. and Toth, P. (1990), *Knapsack Problems: Algorithms and Computer Implementations*, John Wiley & Sons, Chichester, New York.
- Neumaier, A. and Shcherbina, O. (2004), Safe bounds in linear and mixed-integer linear programming, *Mathematical Programming* **99**(2), 283–296.
- Pessoa, A. A., Sadykov, R., Uchoa, E. and Vanderbeck, F. (2020), A generic exact solver for vehicle routing and related problems, *Mathematical Programming* **183**(1), 483–523.
- Pessoa, A., Sadykov, R. and Uchoa, E. (2021), Solving bin packing problems using VRPSolver models, *SN Operations Research Forum* **2**, 20.
- Ryan, D. and Foster, B. (1981), *An integer programming approach to scheduling*, Wren A, ed. Computer Scheduling of Public Transport: Urban Passenger Vehicle and Crew Scheduling (North-Holland, Amsterdam).
- Sadykov, R. and Vanderbeck, F. (2013), Bin packing with conflicts: a generic branch-and-price algorithm, *INFORMS Journal on Computing* **25**(2), 244–255.
- Scheithauer, G. and Terno, J. (1997), Theoretical investigations on the modified integer round-up property for the one-dimensional cutting stock problem, *Operations Research Letters* **20**(2), 93–100.
- Steffy, D. E. and Wolter, K. (2012), Valid linear programming bounds for exact mixed-integer programming, *INFORMS Journal on Computing* **25**(2), 271–284.
- Vance, P. H., Barnhart, C., Johnson, E. L. and Nemhauser, G. L. (1994), Solving binary cutting stock problems by column generation and branch-and-bound, *Computational Optimization and Applications* **3**(2), 111–130.
- Wei, L., Luo, Z., Baldacci, R. and Lim, A. (2020), A new branch-and-price-and-cut algorithm for one-dimensional bin-packing problems, *INFORMS Journal on Computing* **32**(2), 428–443.
- Wunderling, R. (1996), Paralleler und Objektorientierter Simplex-Algorithmus, PhD thesis, Technische Universität Berlin.